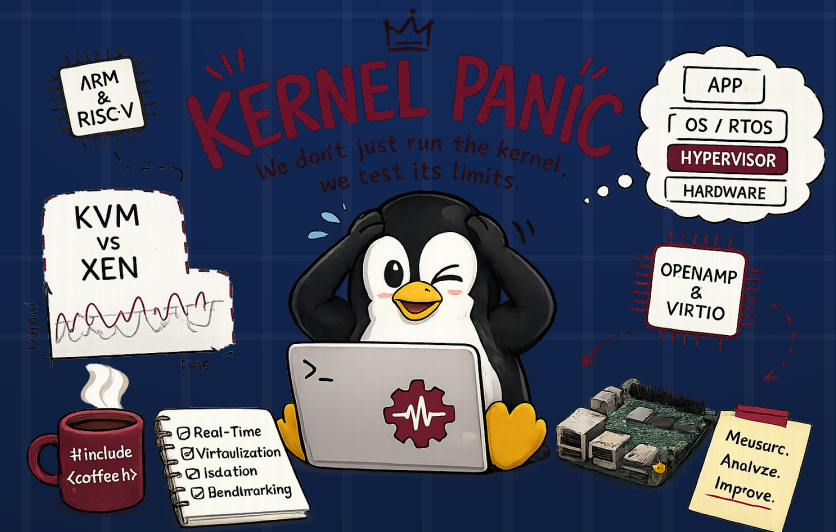


Xen vs KVM under Mixed-Criticality Workloads

An Experimental Characterization



AUTHORS

Rita Marino · Matteo Arnese

PROFESSOR

Ch.mo Prof. Marcello Cinque

SUPERVISORS

Ch.mo Prof De Simone Luigi · Ing. Boccola Francesco

COURSE

Real-Time Systems and Industrial Applications

Agenda

① Motivation & Background

- › Safety-critical systems & hypervisors
- › Related work (Abeni & Faggioli 2019/2020)
- › Our contribution

② Experimental Setup

- › Testbed hardware & kernel configurations
- › OS-level isolation techniques
- › Measurement tools: cyclicttest, TACLe, stress-ng

③ KVM Analysis

- › Baseline, stress, isolation
- › TACLe benchmarks (per-benchmark + summary)

④ Xen Analysis

- › Credit2 vs vCPU Pinning
- › Device Model Priority Inversion
- › 32-scenario stressor study
- › Noisy DomU study
- › TACLe benchmarks & Dom0 stress
- › PV vs PVH vs HVM

⑤ Conclusions

- › KVM vs Xen head-to-head
- › Key takeaways
- › Open questions & future work
- › References

01

Motivation & Background

Why do we need deterministic hypervisors — and what does the prior art say?

MOTIVATION

The Problem: Hard Real-Time in Virtual Machines

Safety-Critical Systems Need Bounded Latency

- › Aerospace, automotive, industrial control — require strict WCET guarantees
- › Consolidating workloads on shared hypervisors reduces cost & complexity
- › **Problem:** co-located guests can temporally interfere with the real-time guest
- › A single missed deadline can be catastrophic in safety-critical contexts

Two Hypervisor Architectures — Different Trade-offs

Xen (Type-1)

Bare-metal hypervisor. Dom0 control domain. HVM / PV / PVH guests. Direct hardware access.

KVM (Type-2)

Hosted hypervisor in Linux. Kernel module. Full hardware emulation via QEMU. PREEMPT_RT-compatible host.



Key question: Can commodity hypervisors guarantee hard real-time bounds for a mixed-criticality guest when sharing the platform with a "noisy" neighbour?

PLATFORM

Critical RT Task
cyclicttest / TACLe

Noisy Guest
stress-ng

same physical CPU →

Hypervisor
Xen 4.17 / KVM

Hardware
AMD · fixed freq

Related Work & Our Contribution

Prior Work — Abeni & Faggioli (2019/2020)

- › Characterized Xen & KVM real-time latency under synthetic stress
- › Found QEMU priority-inversion bug in Xen HVM → unbounded latency
- › Showed Credit2 scheduler introduces structural jitter
- › Stack: Linux 4.x, Xen ≈4.11, older CPUs

Our Extension — Modern Stack

- › Linux 6.18.35, PREEMPT_RT 6.18.35-rt5, Xen 4.17.3
- › TACLe real-world benchmarks (5 programs, up to 1M loops)
- › 32-scenario stressor sweep (cache / interrupts / rawsock)
- › Noisy DomU study (dedicated "neighbour" VM)
- › PV / PVH / HVM comparison

Key Contributions



Verify prior findings on a decade-newer software stack



Confirm priority-inversion fix in Xen 4.17.3



Identify optimal configuration for hard real-time determinism



Reveal nuanced trade-offs in pinning and scheduler choice

Anchor Papers

- › L. Abeni, D. Faggioli — *IEEE ISORC 2019*
- › L. Abeni, D. Faggioli — *JSA 2020*
- › M. Cinque et al. — *JSS 2024*

02

Experimental Setup

Hardware testbed, kernel configurations, OS-level isolation, and measurement methodology.

Testbed & Kernel Configurations

Hardware Testbed

Component	Details
CPU	AMD (MSI board) — fixed frequency (44× ratio)
C-States	Disabled in BIOS (Global C-state Control off)
Turbo/Boost	Precision Boost Overdrive disabled
Hypervisors	Both Xen 4.17.3 & KVM on same host

Kernel Combinations Tested

4 configurations per hypervisor (Host/Dom0 × Guest/DomU kernel)

	NRT Guest / DomU	RT Guest / DomU
NRT Host / Dom0	NRT-NRT	NRT-RT
RT/LL Host / Dom0	RT-NRT	RT-RT / LL-RT ← best

- KVM

Host uses full PREEMPT_RT patch
- XEN

Dom0 uses Low-Latency kernel (PREEMPT_RT incompatible with PV Dom0)

Kernel Versions

Kernel	Version	Used in
Standard NRT	Linux 6.18.35	NRT configs
PREEMPT_RT	6.18.35-rt5	KVM host, RT guests
Low-Latency	6.18.35-rt5-ll	Xen Dom0, LL-NRT

i Both hypervisors tested on the **same physical machine** to eliminate hardware variability. Fixed CPU clock eliminates P-state-induced jitter.

OS-Level Isolation & Measurement Tools

Isolation Techniques Applied

- › `isolcpus` — prevent OS from scheduling tasks on RT cores
- › IRQ affinity steering — route hardware interrupts to housekeeping cores only
- › vCPU-to-pCPU pinning — static core assignment, bypass dynamic scheduler
- › `mlockall` — lock all pages in RAM, prevent page-fault latency spikes
- › Tickless mode (`nohz_full`) — reduce timer interrupt noise on isolated cores
- › RCU callback offloading to housekeeping cores (`rcu_nocbs`)

Measurement — `cyclictest`

- › 5-minute sustained runs per configuration
- › Wakeup interval: 50 μ s, SCHED_FIFO priority 99
- › Histogram threshold: 1000 μ s — overflows count severe events
- › Metrics: Min / Avg / WCET (Max) / Overflow count

TACLe Benchmarks

- › 5 programs from TACLeBench v1.9 suite
- › 10k–1M execution loops, SCHED_FIFO 98, mlockall, affinity=1
- › Compiled with -O2, linked with libpthread + librt

Stress Injection

- › stress-ng — CPU workers + memory workers
- › Small noise: 2-vCPU 4 GB RAM | Big noise: 16-vCPU 16 GB RAM
- › Dom0 stress: 22 CPU + 24 GB vm workers

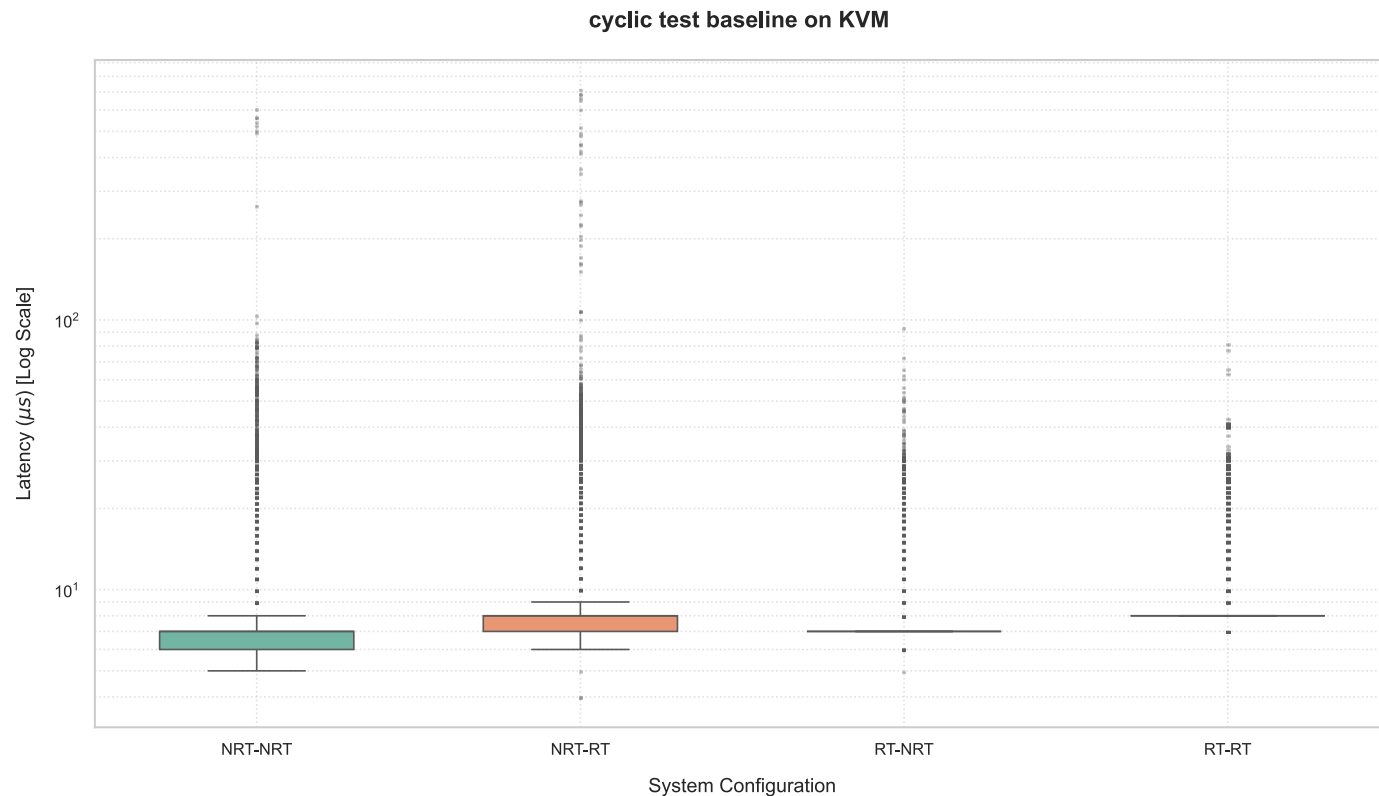
03

KVM

Analysis

Baseline performance, stress conditions, CPU isolation, and TACLe benchmarks on the KVM hypervisor.

KVM: Baseline Performance (No Stress)



Key Observations

✓ **RT-RT (Full Stack):** 81 μs WCET, 0 overflows — 92% of cycles at exactly 8 μs . Near bare-metal behaviour.

i **RT-NRT:** Surprisingly good (93 μs) — RT host protects the guest vCPU scheduling even without a guest-side patch.

⚠ **Determinism is end-to-end** — optimising only one side (host OR guest) is insufficient.

✗ **The NRT-RT anomaly:** An RT guest on a non-RT host achieves a WCET of 6.1 ms — worse than the fully unoptimised NRT-NRT case! A guest-side RT patch is powerless if the host cannot guarantee deterministic vCPU scheduling.

KVM: Baseline Performance (No Stress)

Worst-Case Latencies

Worst-Case Latencies — Baseline Configuration

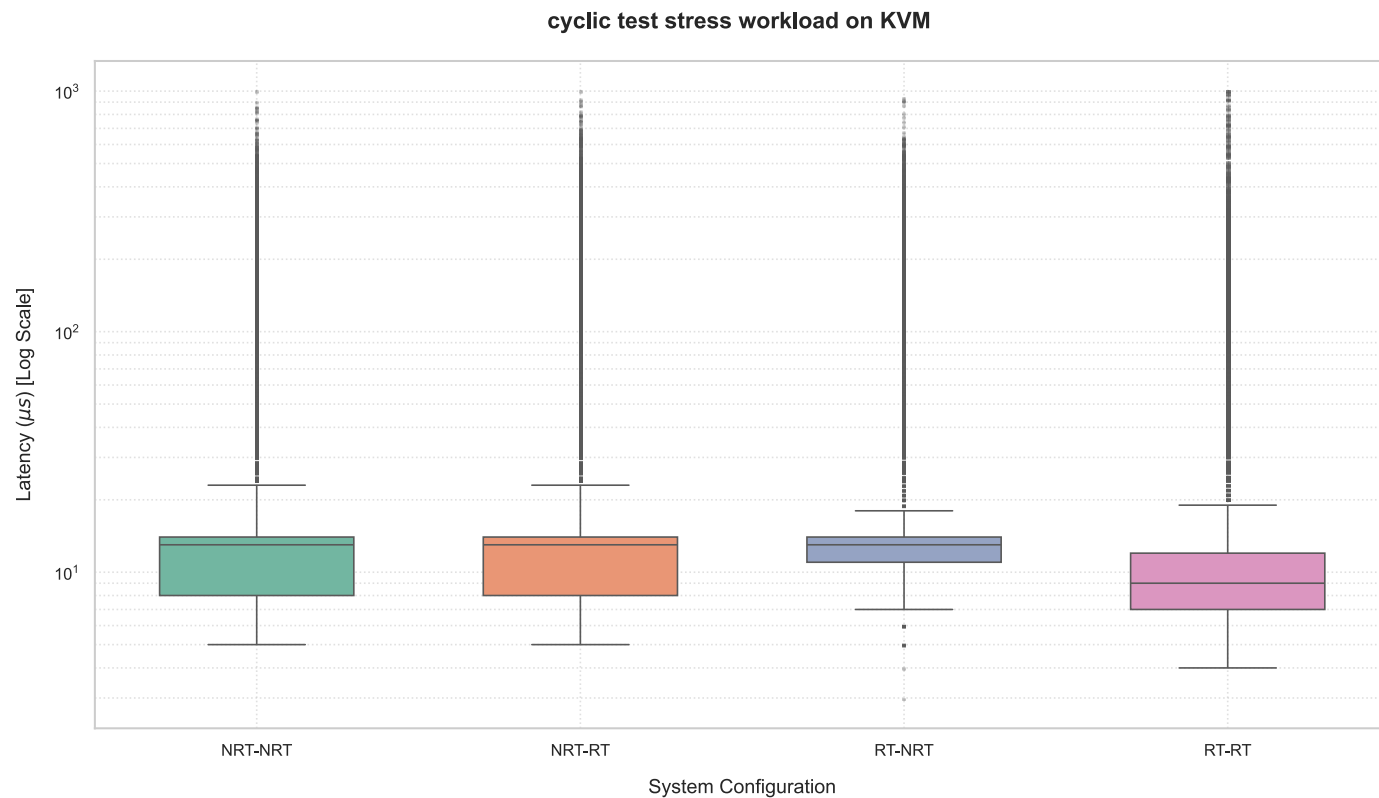
Configuration	Min Latency	Average Latency ± SD	WCET (Max)	Overflows
NRT Host + NRT Guest	5 µs	7.10 ± 2.39 µs	602 µs	0
NRT Host + RT Guest	4 µs	8.55 ± 3.27 µs	6,114 µs ⚠	0
RT Host + NRT Guest	4 µs	7.22 ± 0.98 µs	93 µs	0
RT Host + RT Guest	7 µs	8.15 ± 1.15 µs	81 µs ✓	0

81 µs
BEST WCET (RT-RT)

6,114 µs
WORST WCET (NRT-RT)

KVM Under Stress: Catastrophic Failure

Host-side stress-ng workload introduced (CPU + memory workers)



Key Observations

● **RT-RT under stress:** 283 histogram overflows (latencies >1 ms). Even the best configuration fails to guarantee hard RT bounds under load.

⚠ **RT-NRT percentage jump (+54,000%):** The RT host was very good at baseline (93 μs) — making the stress-induced collapse appear even more dramatic proportionally.

KVM Under Stress: Catastrophic Failure

Worst-Case Latencies

Worst-Case Latencies Under Stress

Config	Stress Average	Stress WCET	Stress Overflows	Average Δ%	WCET Δ%
NRT-NRT	13.60 ± 11.18 μs	44,409 μs	11	+92%	+7,277%
NRT-RT	13.69 ± 12.00 μs	51,069 μs	26	+60%	+735%
RT-NRT	13.33 ± 8.54 μs	50,257 μs	11	+85%	+53,940%
RT-RT	11.84 ± 12.79 μs	9,829 μs	283	+45%	+12,035%

Root Cause Analysis

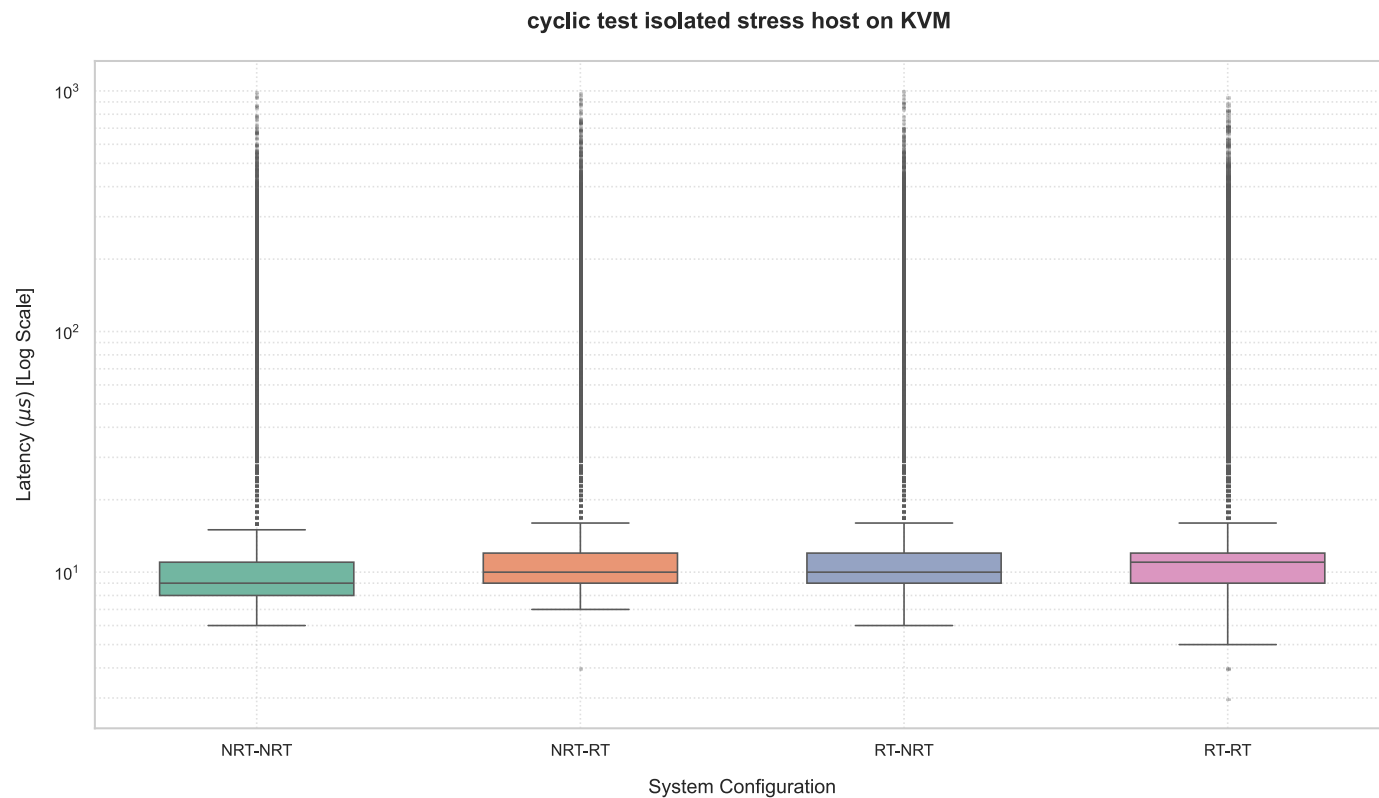
- › Stress workload on host creates massive CPU contention and lock saturation
- › Even PREEMPT_RT cannot prevent non-deferrable hardware interrupt storms
- › Non-preemptible critical sections in kernel spinlocks starve the VM vCPU
- › NRT-RT gets hit worst in absolute terms: guest RT priorities meaningless when host VCPU is starved
- › RT-RT has fewer 50ms spikes but *chronic* millisecond-scale stalls (283 overflows)



Conclusion: KVM with host-side stress cannot guarantee hard real-time bounds in any tested configuration.

KVM: CPU Isolation — Partial Recovery

vCPU pinned to isolated cores (isolcpus + IRQ steering) while host stress runs on remaining cores



Key Observations



Best case (RT-RT + isolated): 1.9 ms
WCET, 14 overflows. Isolation is a *massive* improvement but cannot achieve hard real-time. Residual spikes originate from hardware interrupts and non-preemptible kernel sections that cannot be prevented at the virtualization layer.

KVM: CPU Isolation — Partial Recovery

Worst-Case Latencies

Worst-Case Latencies Under Stress (Isolated)

Config	Min Latency	Average Latency ± SD	WCET (Max)	Overflows
NRT-NRT	6 μs	11 μs	2,024 μs	19
NRT-RT	4 μs	12 μs	2,299 μs	11
RT-NRT	6 μs	13 μs	1,502 μs	16
RT-RT	3 μs	13 μs	1,935 μs	14

-95%

WCET REDUCTION FROM ISOLATION

14

OVERFLOWS IN BEST CASE (RT-RT)

✗ **KVM hard-RT verdict:** Not suitable for applications requiring strict microsecond-level WCET bounds when co-hosting heavy workloads — even with full isolation.

TACLeBench Suite: Real-World Workloads

Run on RT-RT (best KVM config) · 4 scenarios: Baseline / Small Noise / Big Noise / Big Noise + Pinned

matrix1
KERNEL
Raw compute, matrix operations.
Cache-bound. Sub-μs avg execution time.

huff_enc
SEQUENTIAL
Data compression (325 SLOC).
Single-threaded. ~16 μs avg.

lift
APPLICATION
Elevator controller (361 SLOC).
Cyber-physical control loop. ~19 μs avg.

test3
STRESS TEST
Artificial WCET stress (4,235 SLOC). 10k loops. ~8,360 μs avg.

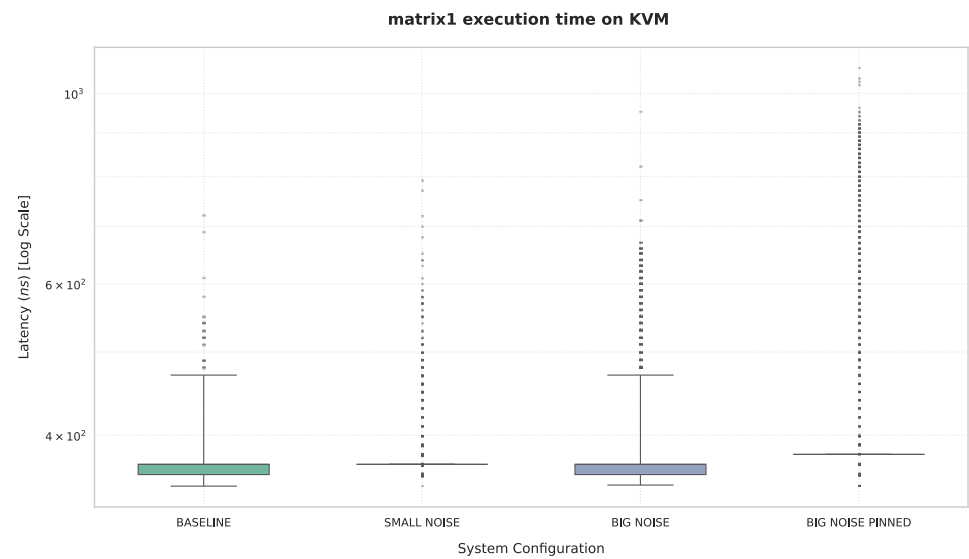
DEBIE
PARALLEL
Aerospace debris monitor (6,615 SLOC, 8 tasks). ~27,000 μs avg.

Test Scenarios (KVM: RT-RT configuration, 4 dedicated host cores)

Scenario	Noisy Guest	Pinning	Purpose
Baseline	None	No	Reference WCET without interference
Small Noise	2 vCPUs, 4 GB RAM	No	Light inter-guest interference
Big Noise	16 vCPUs, 16 GB RAM	No	Heavy inter-guest interference
Big Noise Pinned	16 vCPUs, 16 GB RAM	Yes	Effectiveness of static vCPU allocation under heavy load

 Kernel scheduling isolation (isolcpus) was **not** used for TACLe — it interfered with KVM's vCPU-to-pCPU assignment. IRQ affinity + tickless mode were applied instead.

TACLE: matrix1 — Raw Compute (Kernel Benchmark)



matrix1: Execution Time Across Scenarios

Scenario	Min (μs)	Avg ± SD (μs)	WCET (μs)	Jitter (μs)
Baseline	0	0 ± 0	1	1
Small Noise	0	0 ± 0	0	0
Big Noise	0	0.00 ± 0.01	6	6
Big Noise Pinned	0	0 ± 0	1	1

TACLe: matrix1 — Analysis

Raw compute (kernel benchmark): sensitivity to noise

Analysis

- › matrix1's tiny working set keeps it entirely cache-resident: baseline and small-noise runs are statistically indistinguishable (WCET 0–1 μ s), making it effectively immune to light interference
- › Big Noise is the only scenario that perturbs the task: WCET jumps to 6 μ s (6 $\times$ baseline) as sporadic cache evictions and context-switches briefly interrupt the otherwise instantaneous compute loop
- › Pinning single-handedly reverses the effect, restoring WCET to 1 μ s — confirming the disturbance is scheduling-induced, not compute-bound
- › Compared to DEBIE's multi-task synchronisation overhead, matrix1's single-threaded, cache-bound nature makes it the most naturally resilient kernel benchmark in the suite



matrix1 shows that isolated, cache-resident kernel loops are **nearly immune** to virtualization noise: **only heavy interference** (Big Noise) introduces measurable jitter, and even that is almost entirely reversed by **core pinning**.

6 $\times$

WCET INCREASE UNDER BIG NOISE

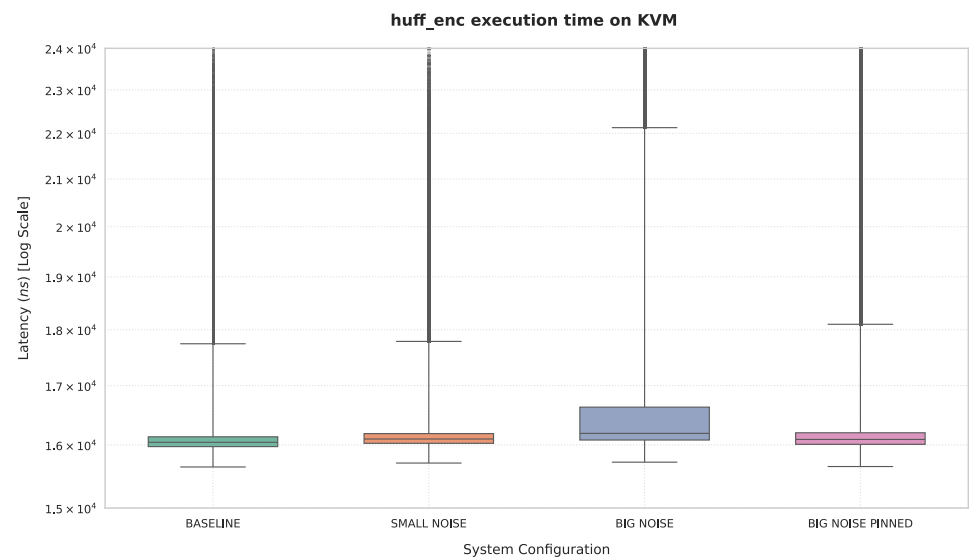
0 μ s

JITTER UNDER SMALL NOISE

–83%

WCET REDUCTION FROM PINNING

TACLe: huff_enc — Data Compression (Sequential)



huff_enc: Execution Time Across Scenarios

Scenario	Min (μs)	Avg ± SD (μs)	WCET (μs)	Jitter (μs)
Baseline	15	16.02 ± 3.39	64	49
Small Noise	15	15.86 ± 3.76	61	46
Big Noise	15	26.21 ± 5.01	69	54
Big Noise Pinned	15	16.53 ± 3.73	62	47

TACLe: huff_enc — Analysis

Data compression (sequential): sensitivity to noise

Analysis

- › huff_enc's sequential, single-pass structure keeps latency remarkably stable at baseline: 16 μ s average, 49 μ s jitter — already higher than matrix1 due to its more complex (non-cache-resident) instruction mix
- › Small Noise has virtually no effect (avg 15 μ s, jitter 46 μ s) — statistically indistinguishable from baseline
- › Big Noise is the only scenario that meaningfully perturbs the average (16→26 μ s, +63%), yet the WCET barely moves (64→69 μ s, +8%) — the disturbance mostly inflates the typical case, not the tail
- › Pinning fully restores baseline behaviour (avg 16 μ s, jitter 47 μ s), confirming the Big Noise effect was scheduling-induced contention, not intrinsic to the algorithm



Average rises significantly under big noise (+63%) but WCET only increases modestly (+8%). Pinning restores the average to baseline.

+63%

AVERAGE LATENCY INCREASE UNDER BIG NOISE

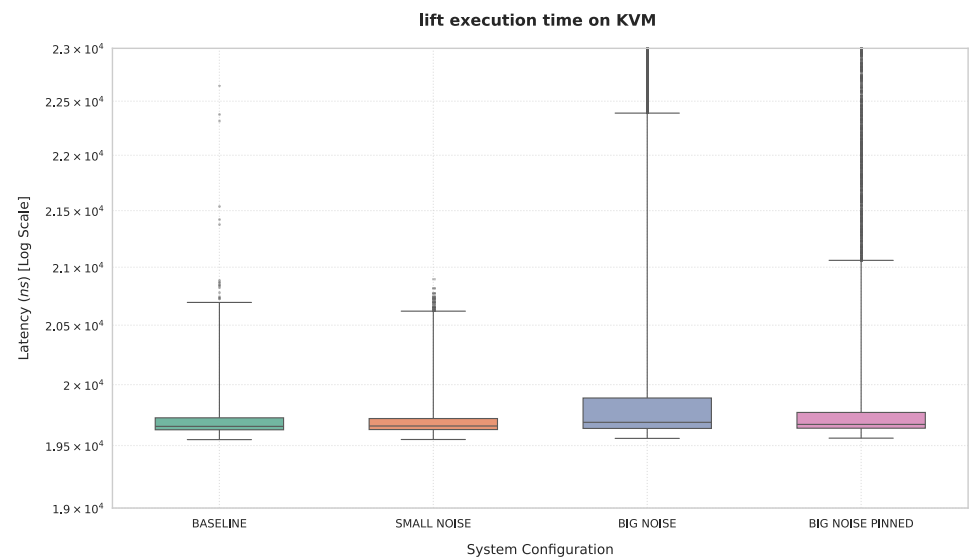
+8%

WCET INCREASE UNDER BIG NOISE

100%

BASELINE RESTORED WITH PINNING

TACLe: lift — Elevator Controller (Application)



lift: Execution Time Across Scenarios

Scenario	Min (μs)	Avg ± SD (μs)	WCET (μs)	Jitter (μs)
Baseline	19	19.44 ± 3.00	61	42
Small Noise	19	19.44 ± 2.99	58	39
Big Noise	19	19.42 ± 3.00	280 ⚠	261
Big Noise Pinned	19	20.31 ± 4.91	67	48

TACLe: lift — Analysis

Elevator controller (application): sensitivity to noise

Analysis

- › lift shows strong determinism at baseline: 19 μ s average (= minimum), WCET 61 μ s, jitter 42 μ s — a tightly bounded control task
- › Small Noise barely changes behaviour (WCET 58 μ s, jitter 39 μ s) — the task is resilient to minor disturbances
- › Big Noise causes a sharp breakdown in determinism: WCET spikes to 280 μ s (4.6 $\times$ baseline), jitter balloons to 261 μ s — even highly repetitive, lightweight control tasks suffer severe, occasional micro-interruptions under heavy KVM load
- › Pinning fully restores strict determinism (avg 20 μ s, WCET 67 μ s, jitter 48 μ s) — completely eliminating the 280 μ s spike seen under stress

● **Dangerous tail growth:** Average stays flat at 19 μ s but WCET jumps 359% to 280 μ s under big noise. **Pinning completely eliminates the spike** (67 μ s). Classic "average misleads, WCET matters" case.

4.6 $\times$

WCET DEGRADATION UNDER BIG NOISE

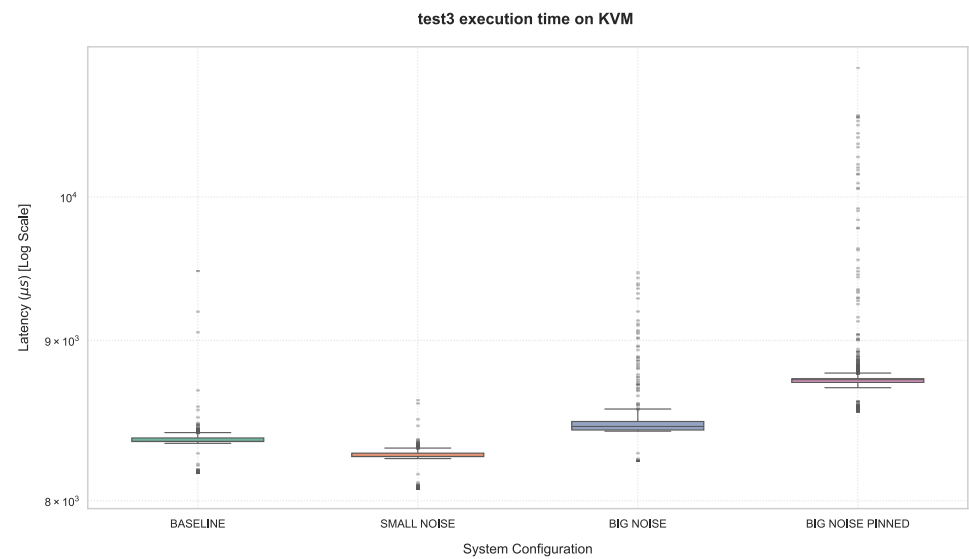
261 μ s

JITTER UNDER BIG NOISE

100%

SPIKE ELIMINATED WITH PINNING

TACLE: test3 — WCET Stress Test



test3: Execution Time Across Scenarios

Scenario	Min (μs)	Avg ± SD (μs)	WCET (μs)	Jitter (μs)
Baseline	8,169	8,364 ± 32	9,471	1,302
Small Noise	8,078	8,270 ± 25	8,612	534
Big Noise	8,243	8,458 ± 52	9,458	1,215
Big Noise Pinned	8,544	8,745 ± 87	10,996	2,452

TACLe: test3 — Analysis

WCET stress test: sensitivity to noise

Analysis

- › test3's baseline already carries substantial jitter (1,302 μ s) — a medium-weight computational loop naturally more exposed to scheduling noise than lightweight kernels like matrix1
- › Small Noise paradoxically tightens the jitter (534 μ s, the lowest of the series) — a statistical effect of the specific interleaving, not a sign of added protection
- › Big Noise remains close to baseline (1,215 μ s jitter) — this loop's medium duration appears to average out short bursts of interference
- › Pinning is counter-productive here: WCET rises to 10,996 μ s and jitter more than doubles baseline (2,452 μ s) — the highest of all four scenarios

● Unlike every other TACLe benchmark, **pinning does not help test3** — it produces the worst WCET (10,996 μ s) and worst jitter (2,452 μ s) of the series. Confining a medium-weight, longer-running loop to isolated cores may **starve it of the scheduling flexibility it needs**, or concentrate it alongside residual system interrupts on that core.

1,302 μ s

BASELINE JITTER

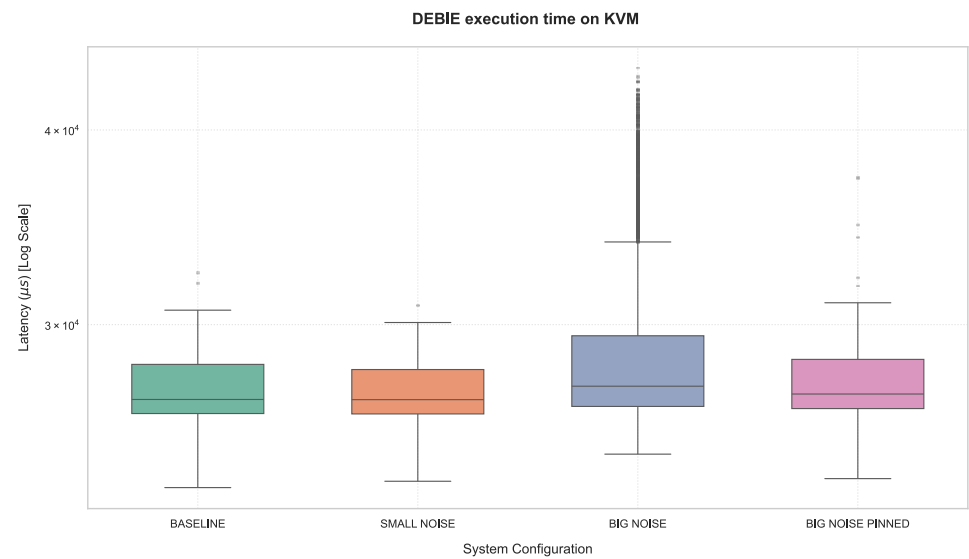
−59%

JITTER IMPROVEMENT UNDER SMALL NOISE

+88%

JITTER DEGRADATION WITH PINNING

TACLE: DEBIE — Aerospace Multi-Task Parallel Benchmark



DEBIE: Execution Time Across Scenarios

Scenario	Min (μs)	Avg ± SD (μs)	WCET (μs)	Jitter (μs)
Baseline	23,578	27,214 ± 1,304	32,409	8,831
Small Noise	23,800	27,221 ± 1,305	30,872	7,072
Big Noise	24,773	28,846 ± 3,814	43,849	19,076
Big Noise Pinned	23,893	27,444 ± 1,328	37,284	13,391

TACLe: DEBIE — Analysis

Multi-task parallel workload: sensitivity to noise

Analysis

- › DEBIE's 8-task parallel structure makes it inherently sensitive to preemption and synchronisation overhead
- › Big Noise nearly **doubles the jitter** (8,831 → 19,076 μ s)
- › Small noise actually *reduces* WCET vs baseline — stochastic effect of scheduling
- › Pinning helps (37,284 μ s vs 43,849 μ s) but DEBIE's variance cannot be eliminated — inter-task synchronisation overhead persists



Multi-task parallel benchmarks show a fundamentally different sensitivity profile than single-threaded tasks: the average grows with noise, and pinning provides only partial mitigation.

+35%

WCET INCREASE (BIG NOISE)

+6%

AVERAGE INCREASE (BIG NOISE)

-14%

WCET REDUCTION (PINNING)

KVM TACLe: Max Execution Time Summary (μs)

Benchmark	Baseline	Small Noise	Big Noise	Big Noise Effect	Big Noise Pinned	Pinning Effect
DEBIE	32,409	30,872	43,849	+35%	37,284	-15%
huff_enc	64	61	69	+8%	62	-10%
lift	61	58	280 ⚠	+359%	67	-76%
matrix1	1	0	6	+500%	1	-83%
test3	9,471	8,612	9,458	-0.14%	10,996	+16%

✔ **Pinning is highly effective for short-lived tasks:** lift WCET reduced 76% (280 → 67 μs), matrix1 restored to baseline. lightweight control-loop tasks benefit most.

⚠ **Pinning can worsen long-running tasks:** test3 WCET increases +16% under pinning — concentrated interference on a single core creates an isolated spike larger than distributed scheduling would.

💡 **Core insight:** The impact of noise and the benefit of pinning are highly workload-dependent. Average execution time is a poor proxy for worst-case determinism — WCET must be measured directly.

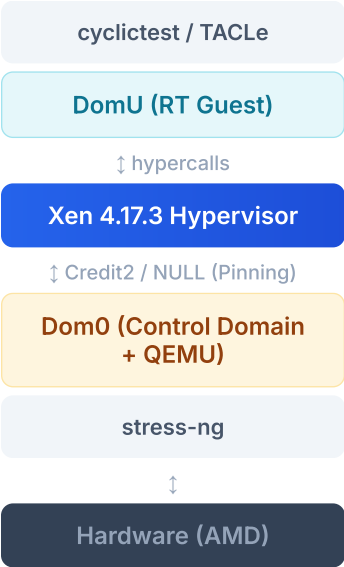
04

Xen Analysis

Scheduler comparison, priority inversion study, 32-scenario sweep, noisy DomU, TACLe benchmarks, and PV/PVH/HVM evaluation.

Xen: Architecture & Tested Configurations

XEN 4.17 STACK



Configurations Tested (cyclictest campaign)

Dom0 Kernel	DomU Kernel	Scheduler	Stress?
NRT	NRT / RT	Credit2	No / Yes
Low-Latency (LL)	NRT / RT	Credit2	No / Yes
NRT	NRT / RT	vCPU Pinning (NULL equiv.)	No / Yes
Low-Latency (LL)	NRT / RT	vCPU Pinning (NULL equiv.)	No / Yes

Credit2 Scheduler

- › Default Xen scheduler
- › Fair-share, work-conserving
- › Dynamic vCPU-to-pCPU mapping
- › Introduces inherent ~32 μs avg jitter

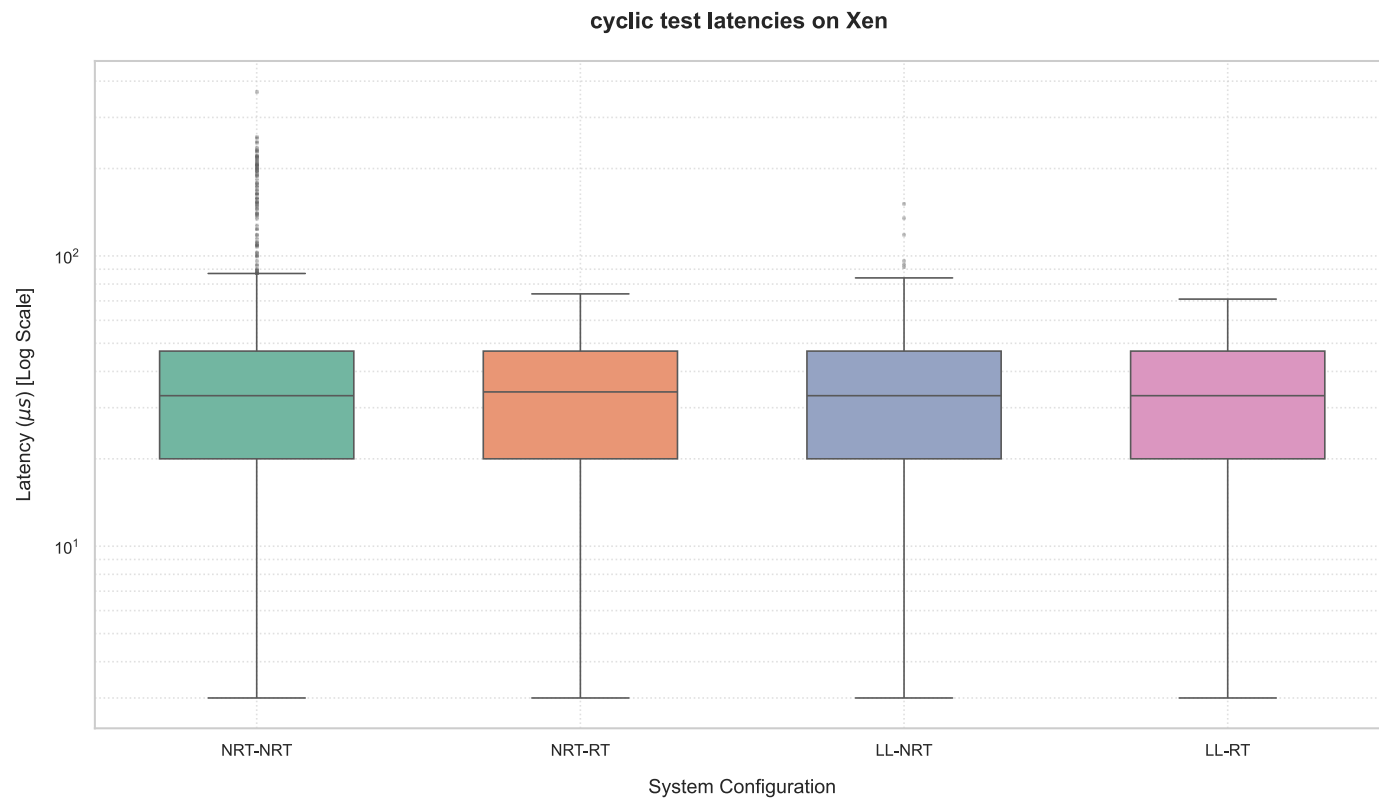
vCPU Pinning (NULL Emulation)

- › NULL scheduler unavailable in Xen 4.17.3 (experimental)
- › Static pinning: Dom0 → first N pCPUs, DomU → last M pCPUs
- › Effectively bypasses Credit2 decisions

⚠ **Key difference from KVM:** Xen Dom0 is a PV guest — PREEMPT_RT patch causes soft CPU lockups in PV mode. Dom0 uses a **Low-Latency** tuned kernel instead.

Xen Baseline: Credit2 Scheduler

No stress workload. All configurations, 5-minute cyclicttest runs.



Key Observations

i Hypervisor jitter is constant: Average latency is $\sim 32 \mu\text{s}$ across all configurations — a structural Xen virtualization constant that Credit2 alone cannot reduce.

✓ RT DomU dominates: NRT-RT achieves $74 \mu\text{s}$ — close to LL-RT. The RT guest successfully bounds its own critical sections.

⚠ Optimizing the DomU matters more than optimizing Dom0: a Real-Time guest alone recovers most of the achievable WCET improvement ($\sim 80\%$), while a Low-Latency Dom0 alone recovers less ($\sim 59\%$) — and combining both yields only a small additional gain ($\sim 4\%$) over the RT DomU alone.

Xen Baseline: Credit2 Scheduler

Worst-Case Latencies

Analysis

- › RT-NRT is the true baseline: with no optimization anywhere in the stack, Credit2's fair-share scheduling produces the widest WCET of the series (369 μ s) — confirming that general-purpose scheduling alone cannot bound worst-case latency
- › Host-side LL tuning helps, but less: a Low-Latency Dom0 alone (LL-NRT, no RT DomU) reduces WCET by 59% (369→152 μ s) — meaningful, but well short of what guest-side RT patching achieves on its own
- › Combining both gives the best, but diminishing, return: LL-RT reaches the series minimum (71 μ s), only 4% better than NRT-RT alone — most of the achievable gain already comes from the DomU side; Dom0 tuning mainly refines an already-good result

Worst-Case Latencies — Credit2 Baseline

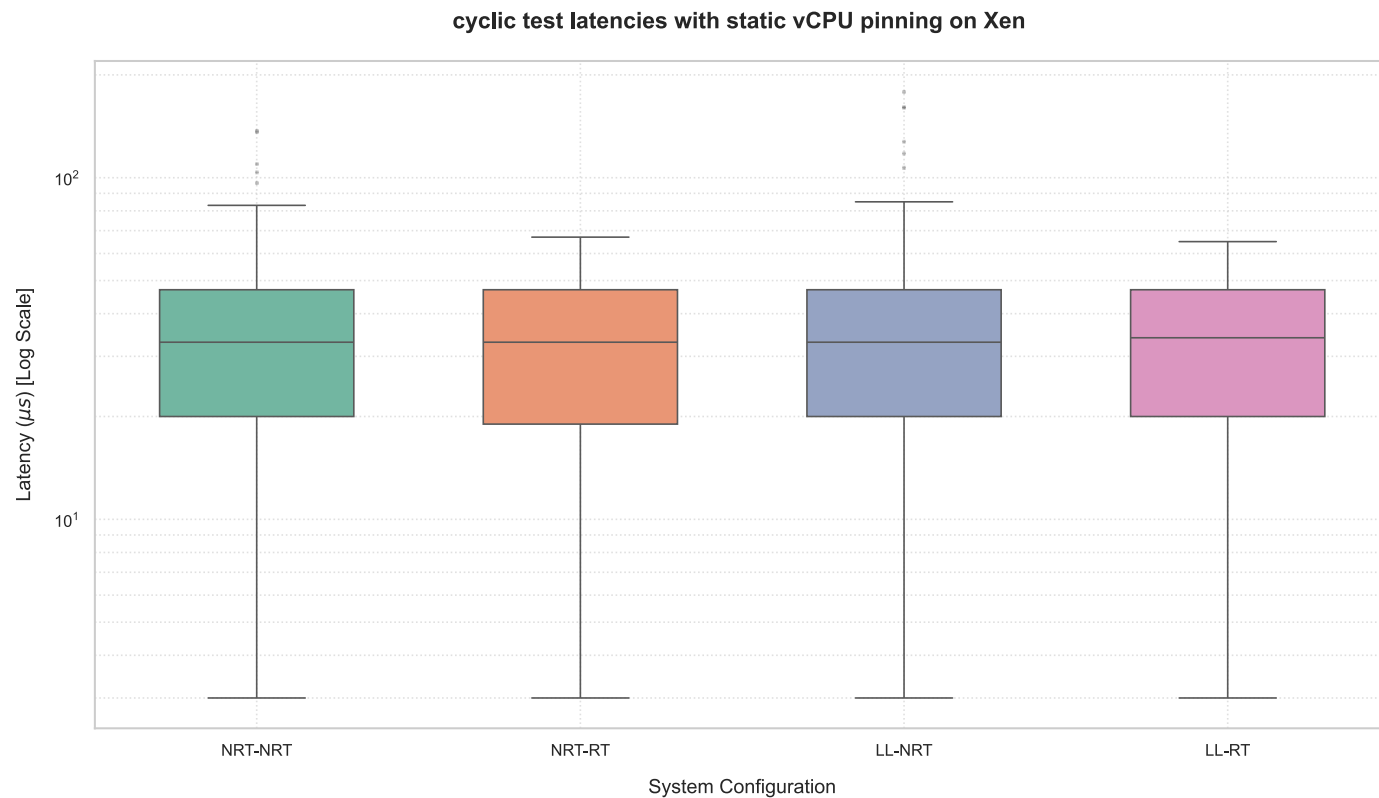
Configuration	Min Latency	Average Latency $\pm$ SD	WCET (Max)	Overflows
NRT-NRT	3 μ s	32.87 $\pm$ 15.17 μ s	369 μ s	0
NRT-RT	3 μ s	33.20 $\pm$ 15.12 μ s	74 μ s	0
LL-NRT	3 μ s	32.71 $\pm$ 15.17 μ s	152 μ s	0
LL-RT	3 μ s	32.89 $\pm$ 15.14 μ s	71 μ s	0

71 μ s
BEST WCET (LL-RT)

369 μ s
WORST WCET (NRT-NRT)

Xen Baseline: vCPU Pinning

No stress workload. All configurations, 5-minute cyclicttest runs.



Key Observations

⚠ LL-NRT pinning anomaly: Pinning worsens WCET for NRT DomU on LL Dom0 (152→179 μs). LL Dom0 optimizations may concentrate certain interrupt patterns on guest-used cores.

✓ LL-RT is best: 65 μs WCET — excellent for a baseline, comparable to KVM RT-RT (81 μs).

i vCPU pinning under Credit2 is used here as a practical substitute for Xen's Null Scheduler, which could not be enabled in this version. Results should be read as "near-static" rather than fully offline scheduling — some residual Credit2 jitter may remain.

Xen Baseline: vCPU Pinning

Worst-Case Latencies

Analysis

- › Pinning is a workaround, not the intended configuration: the Null Scheduler (a purely static, offline allocator) could not be enabled — Xen 4.17.3 rejected it and CPU-pool creation failed, likely due to the scheduler's experimental status in this version. vCPU pinning under Credit2 was used as a practical emulation of the same static-allocation behaviour
- › This explains the residual jitter: because Credit2 remains technically active underneath the pinning configuration, some latent scheduling overhead from Credit2's internal logic may still leak through — the pinned results approximate, but are not strictly identical to, a true offline/static scheduler
- › The close match to expected values validates the approach: the pinned WCETs (65–179 µs) are close enough to the theoretical Null Scheduler behaviour described in the reference methodology to confirm pinning is a reasonable proxy, even if not a perfect one

Worst-Case Latencies — vCPU Pinning

Configuration	Min Latency	Average Latency ± SD	WCET (Max)	Overflows
NRT-NRT	3 µs	32.65 ± 15.17 µs	137 µs	0
NRT-RT	3 µs	32.59 ± 15.16 µs	67 µs	0
LL-NRT	3 µs	32.49 ± 15.19 µs	179 µs	0
LL-RT	3 µs	33.09 ± 15.18 µs	65 µs	0

65 µs

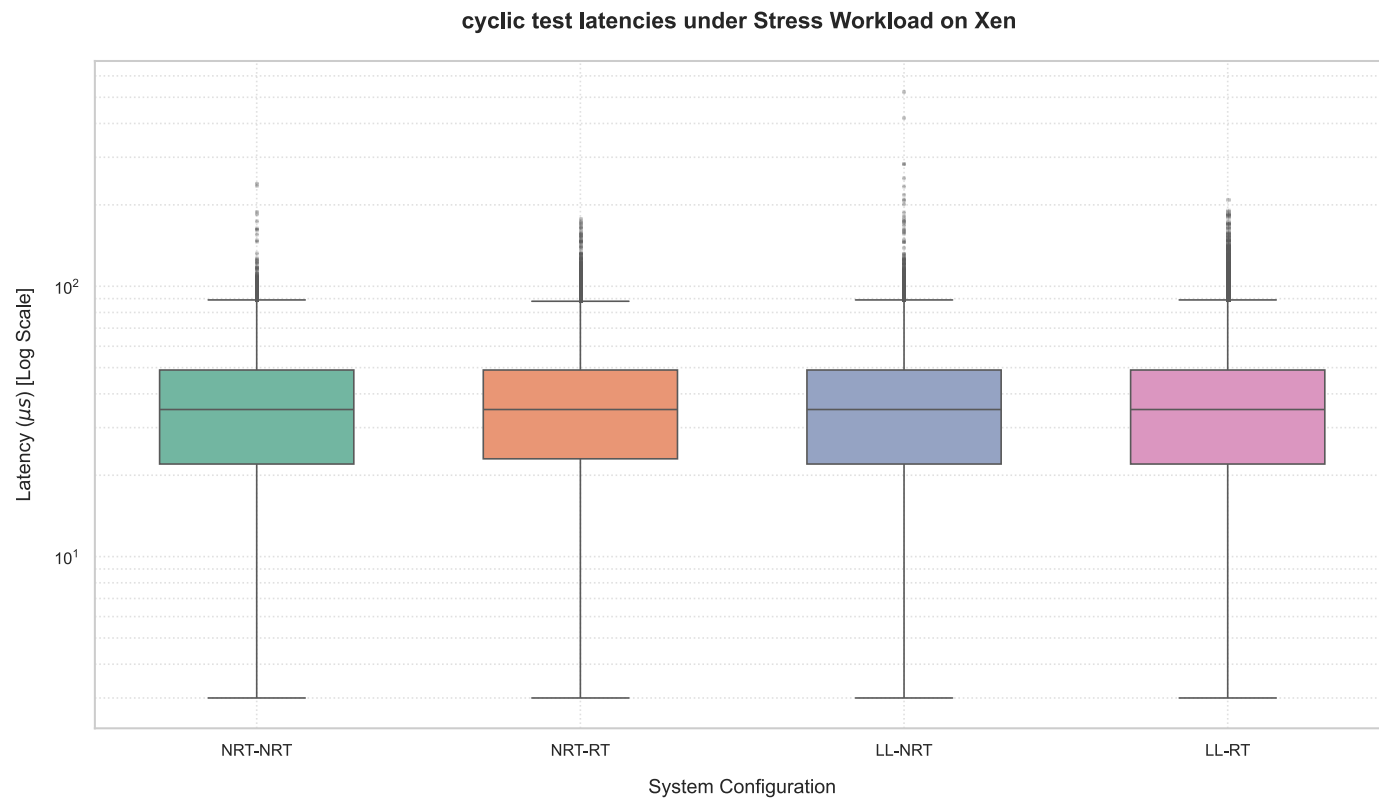
BEST WCET (LL-RT PINNED)

–63%

MAX IMPROVEMENT FROM
UNPINNED (NRT-NRT)

Xen Under Stress: Credit2 Scheduler

Dom0 stress workload applied (stress-ng CPU + memory)



Key Observations

- **LL-NRT Credit2 Stress:** 526 μs — worse than NRT-NRT (239 μs)! Low-latency Dom0 tuning without a corresponding RT DomU can *concentrate* interference and worsen the tail.

Xen Under Stress: Credit2 Scheduler

Worst-Case Latencies

Analysis

- › The ~35 µs average is unshakeable: even under sustained Dom0 stress-ng load, average latency stays pinned at 35 µs across all four configurations — confirming this is Xen's structural virtualization floor, not a function of load or kernel tuning
- › RT DomU is the only reliable shield: whether paired with a standard or Low-Latency Dom0, a PREEMPT_RT guest consistently bounds WCET under stress (177 µs / 209 µs) — proving that guest-side real-time patching is the dominant protective factor, robust to what happens on the host side
- › Optimizing the wrong side backfires: this is the clearest evidence in the dataset that Dom0-only tuning is not a substitute for DomU-side RT patching — and can be counter-productive if applied in isolation
- › Ranking under stress: NRT-RT (177 µs) ≈ LL-RT (209 µs) ≪ NRT-NRT (239 µs) ≪ LL-NRT (526 µs) — the presence of an RT DomU matters roughly 3× more than the presence of an LL Dom0

Worst-Case Latencies Under Stress — Credit2

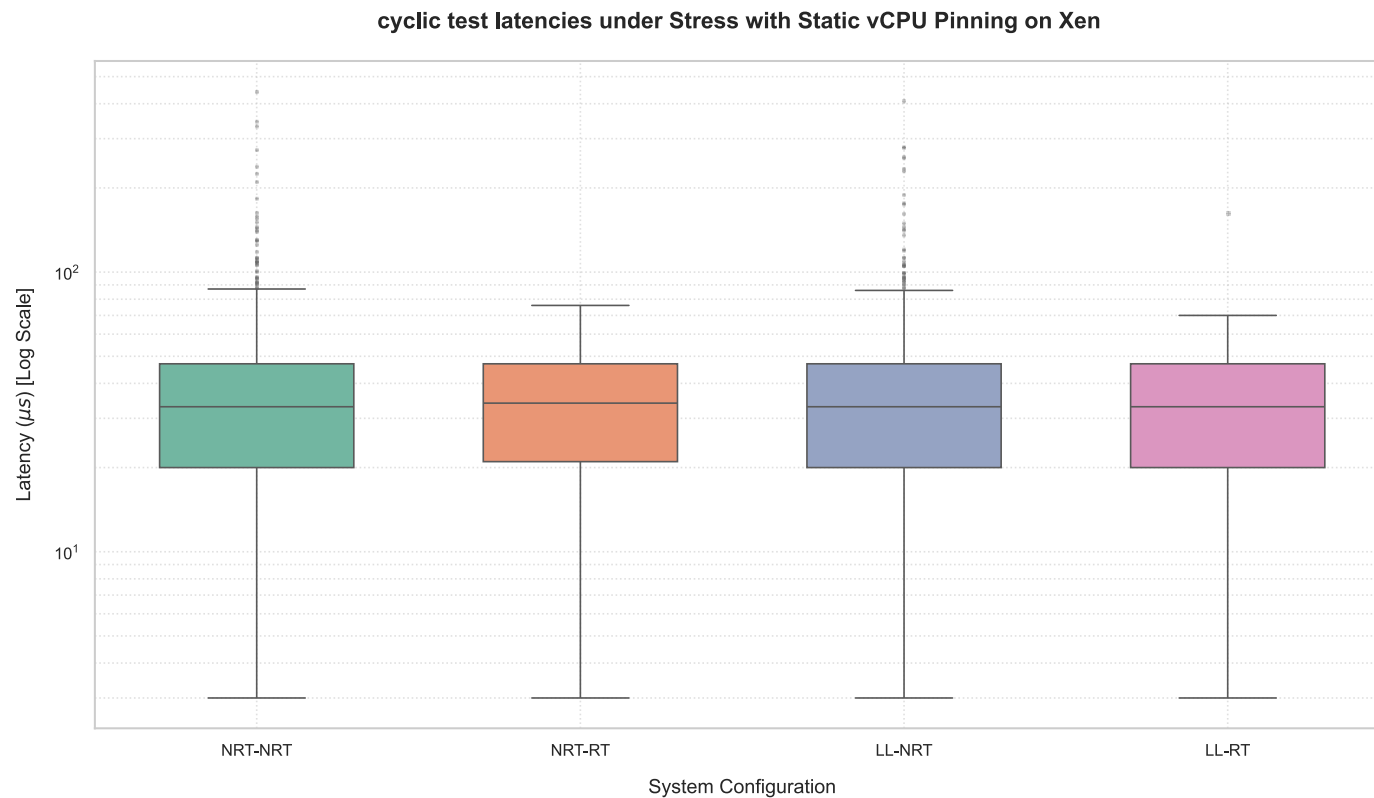
Config	Stress Average	Stress WCET	Overflows	Average Δ%	WCET Δ%
NRT-NRT	35.52 ± 15.50 µs	239 µs	0	+8%	-35%
NRT-RT	35.86 ± 15.54 µs	177 µs	0	+8%	+139%
LL-NRT	35.58 ± 15.55 µs	526 µs	0	+9%	+246%
LL-RT	35.66 ± 15.60 µs	209 µs	0	+8%	+194%

526 µs
WORST WCET (LL-NRT)

177 µs
BEST WCET (NRT-RT)

Xen Under Stress: vCPU Pinning

Dom0 stress workload applied (stress-ng CPU + memory)



Key Observations

★ **NRT-RT + Pinning Stress: 76 μs** — remarkably robust. Only 13% above its unstressed baseline. The RT DomU protects itself even without an LL Dom0.

💡 **Key insight:** The *combination* matters more than the individual optimization. An RT DomU is the single most protective factor. Pinning under stress without an RT DomU (NRT-NRT Pinning Stress: 443 μs) is still unreliable.

Xen Under Stress: vCPU Pinning

Worst-Case Latencies

Analysis

- › RT DomU remains the strongest shield even with pinning: NRT-RT under pinning + stress caps WCET at just 76 µs — the lowest of the four pinned-stress configurations, confirming that guest-side RT patching dominates regardless of scheduling strategy (dynamic Credit2 or static pinning)
- › Pinning only partially rescues the LL-NRT anomaly: 526 µs (Credit2, no pinning) improves to 410 µs with pinning — a 22% reduction, but still the worst result among all pinned-stress configurations, well above even the unoptimized NRT-NRT baseline (443 µs is close, but NRT-NRT's own pinning story is different — see below)
- › Dom0 tuning without DomU tuning remains the weak link: across both scheduling strategies (Credit2 and static pinning), LL-NRT is consistently the worst or near-worst performer under stress — reinforcing that Low-Latency Dom0 tuning cannot compensate for the absence of RT optimization in the DomU, and pinning alone does not fully correct this asymmetry
- › The ~32–33 µs average floor persists even with static pinning: confirming this jitter originates in Xen's structural virtualization overhead itself, not in scheduler choice — pinning bounds the tail (WCET) but does not touch the average

Worst-Case Latencies Under Stress (Pinned)

Config	Stress Average	Stress WCET	Overflows	Average Δ%	WCET Δ%
NRT-NRT	32.76 ± 15.10 µs	443 µs	0	+0.34%	+223%
NRT-RT	33.23 ± 15.09 µs	76 µs	0	+1.96%	+13%
LL-NRT	32.83 ± 15.06 µs	410 µs	0	+1.05%	+129%
LL-RT	32.73 ± 15.09 µs	163 µs	0	-1.11%	+150%

76 µs
BEST WCET (NRT-RT)

443 µs
WORST WCET (NRT-NRT)

QEMU Priority Inversion: A Historical Bug — Now Fixed

Historical Issue (Abeni & Faggioli 2019)

- › HVM DomU depends on QEMU Device Model running in Dom0
- › Under Dom0 stress: QEMU (low-priority) gets preempted
- › RT DomU blocked waiting for QEMU → priority inversion → **unbounded latency spikes**
- › Mitigation: elevate QEMU to SCHED_FIFO priority 99

Our Investigation — Xen 4.17.3 (3 Experiments)

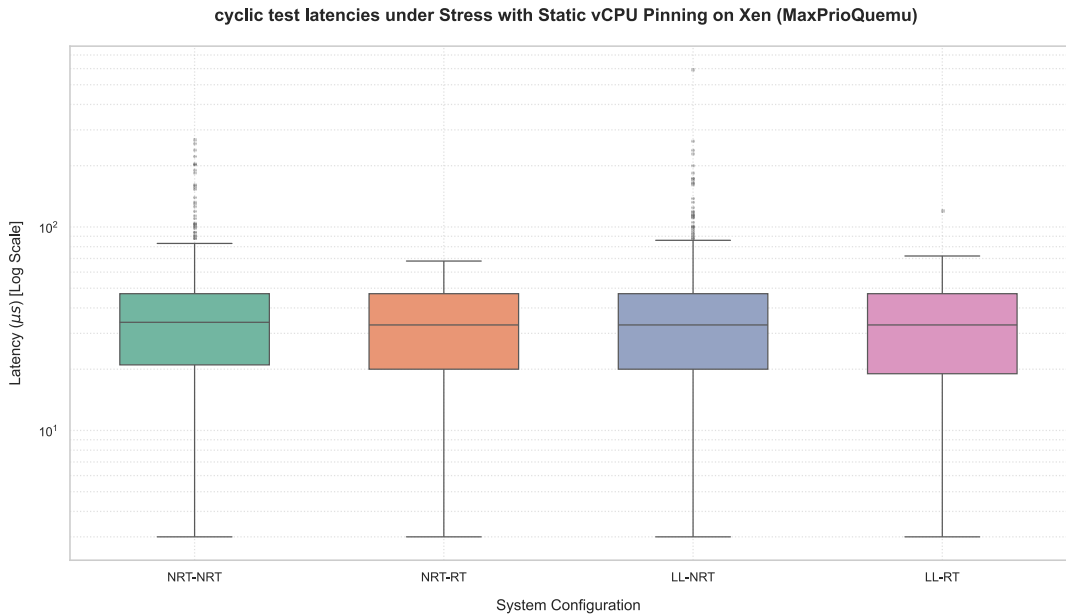
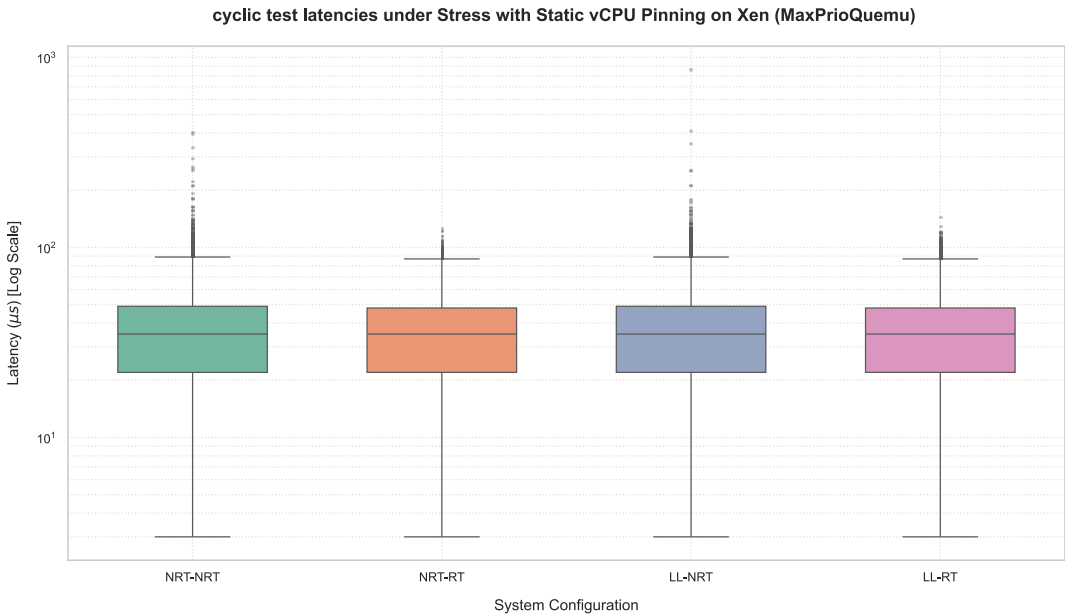
Experiment	QEMU Priority	Result
1 — Default QEMU	Default	No extreme outliers. Comparable to baseline.
2 — Max QEMU (NULL)	SCHED_FIFO 99	No improvement. Distributions virtually identical.
3 — Max QEMU (Credit2)	SCHED_FIFO 99	Same result — no beneficial effect.

Explanation

- › Modern Xen HVM **decouples essential timer/interrupt delivery** from the QEMU Device Model
- › CPU-bound real-time workloads (cyclicttest) exercise timer wakeups — no complex I/O requiring QEMU intervention
- › DomU can maintain temporal constraints via **hardware virtualization extensions alone**
- › Manually elevating QEMU priority provides no benefit and is unnecessary

✓ **Conclusion:** The priority inversion bug has been **FIXED in Xen 4.17.3**. One of the major historical concerns about Xen HVM is no longer a practical obstacle on the modern software stack.

QEMU Priority Inversion: A Historical Bug — Now Fixed



32-Scenario Stressor Study: Summary

4 factors × combinations: Dom0 kernel (NRT/LL) · vCPU pinning (yes/no) · Guest type (nrt/rt5) · Stressor (baseline/cache/interrupts/rawsock)

Summary by Pinning Effect (all 32 scenarios)

Group	Avg Mean Latency	Max WCET	Total Overflows	Runs with Overflow
Pinned guests	32.6 µs	35,984 µs*	48	2 / 15
Unpinned guests	35.4 µs	61,826 µs	179	8 / 16

* Single isolated anomaly — one outlier at ~36 ms, cycle 913 of 4.93M

4×

FEWER RUNS WITH OVERFLOWS (PINNED VS UNPINNED)

88

OVERFLOWS — WORST SINGLE RUN (NRT + NRT + CACHE)

Key Findings

- ① **vCPU pinning is the dominant factor** — almost deterministic behaviour regardless of Dom0 kernel or stressor type
- ② **Unpinned nrt-hvm guest is most fragile** — up to 62 ms WCET, 88 overflows under cache stressor
- ③ **rt5 guest systematically more stable** than nrt guest, whether pinned or not
- ④ **cache stressor most unpredictable** — worst single case (88 overflows) but harmless with pinning (0 overflows)
- ⑤ **LL vs NRT Dom0:** No clear winner on unpinned guests — interaction with guest type and stressor is more complex

Experimental Design: 4 Factors, 32 Combinations

The Four Factors

Dom0 Kernel

Standard ("NRT") vs. Low-Latency/PREEMPT_RT ("LL")

vCPU Pinning

Guests pinned to dedicated physical cores vs. unpinned (free Xen scheduling)

Guest Type

Standard Linux ("nrt") vs. RT-patched kernel ("rt5")

Dom0 Stressor

none (baseline) · cache · interrupts · raw sockets — on top of always-on CPU+memory load

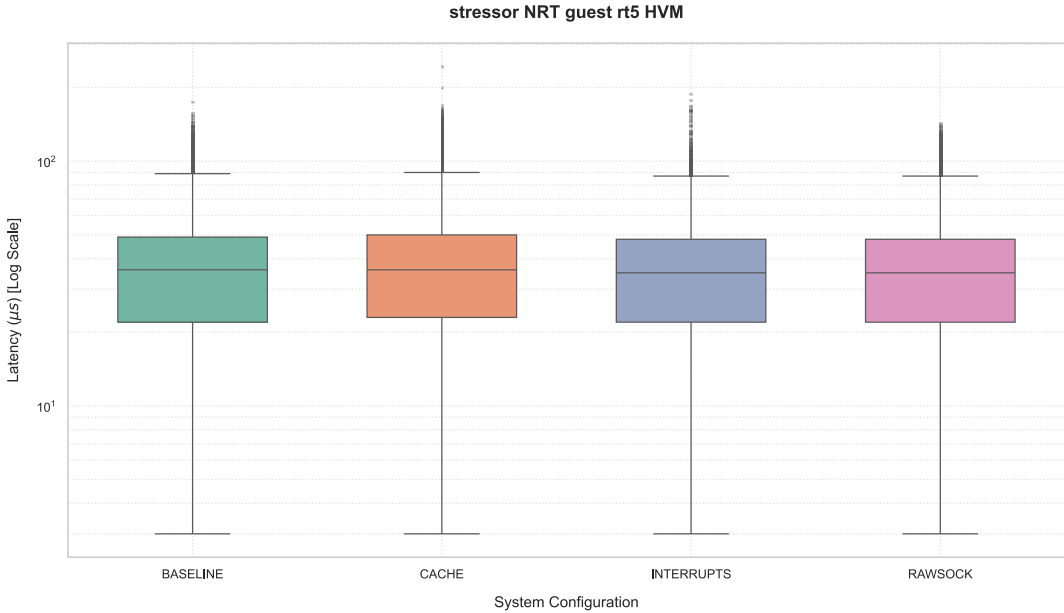
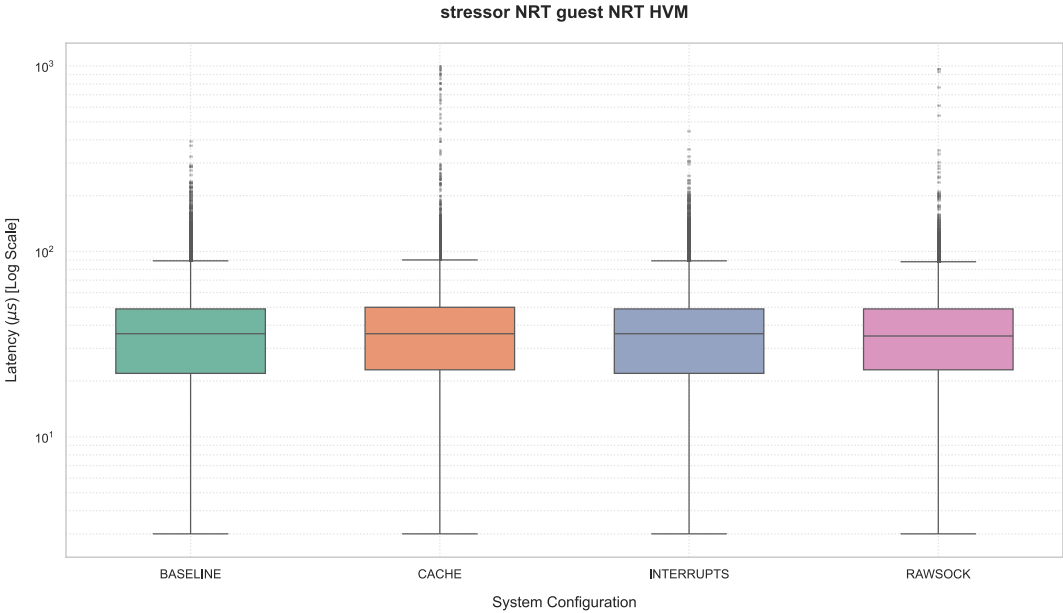
Protocol per Scenario

- › Always-on stressors started on Dom0: `stress-ng --cpu 22, --vm 12 --vm-bytes 2G`, held for the guest's entire test block
- › Scenario-specific sequential stressor added (none / `--cache` / `--interrupts` / `--rawsock 22`)
- › DomU created, 30 s settling time, then `cyclicttest` runs for 5 minutes (priority 99, 50 μ s interval, 1000 μ s histogram threshold)
- › DomU destroyed, histogram retrieved, all stressors terminated, Dom0 caches cleared before the next scenario

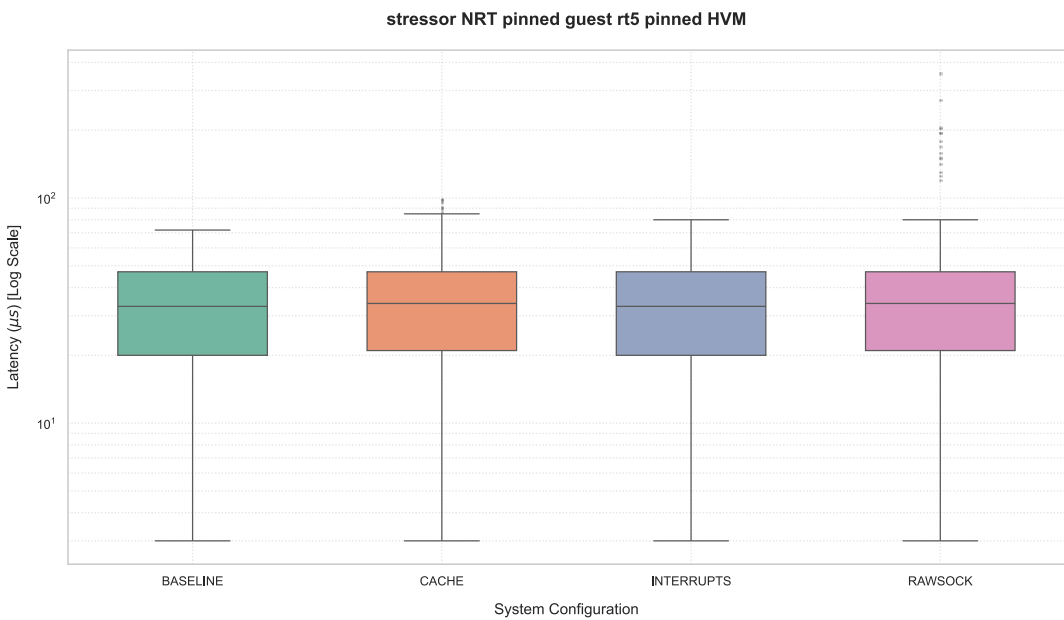
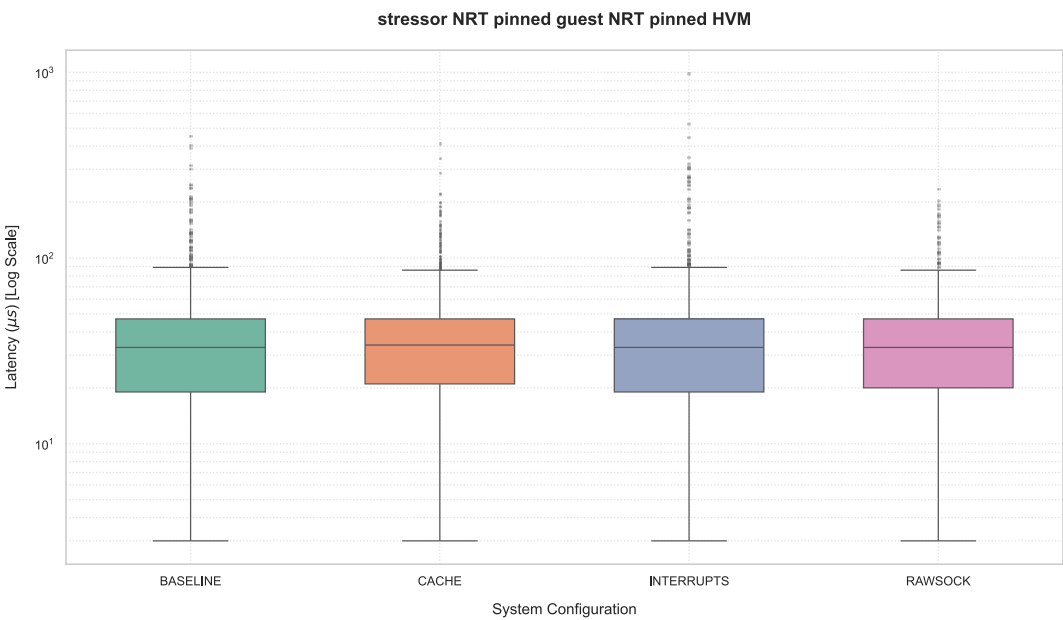


Each of the 32 kernel × guest × stressor combinations was executed in a **single run** (no repetitions) — isolated anomalies (see later slide) cannot be statistically distinguished from transient noise with this design.

32-Scenario Stressor Study: NRT Dom0



32-Scenario Stressor Study: NRT Dom0 (Null Scheduler)

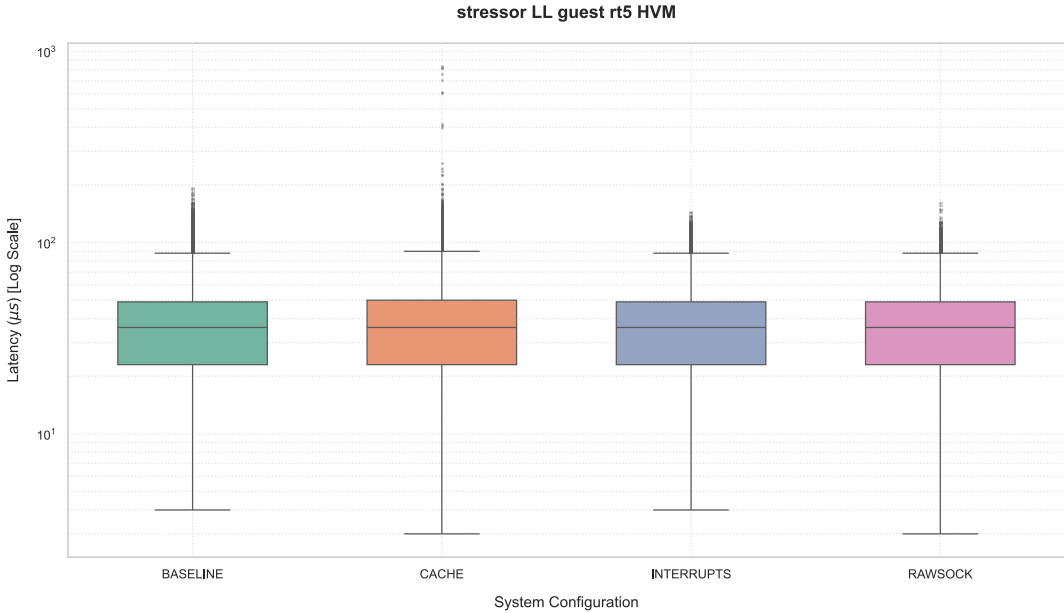
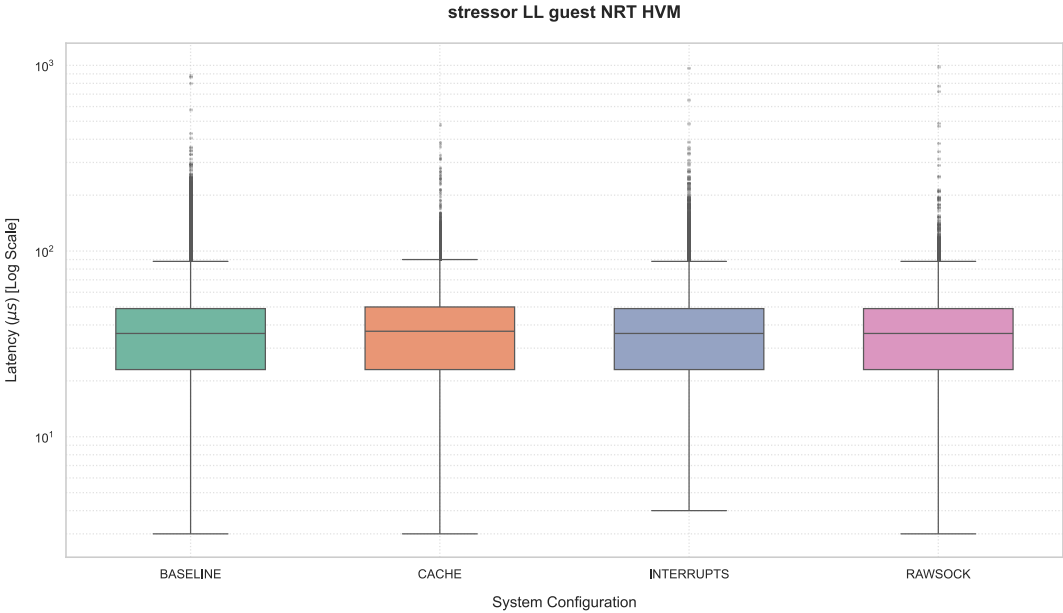


Full Results — Standard (NRT) Dom0 Kernel

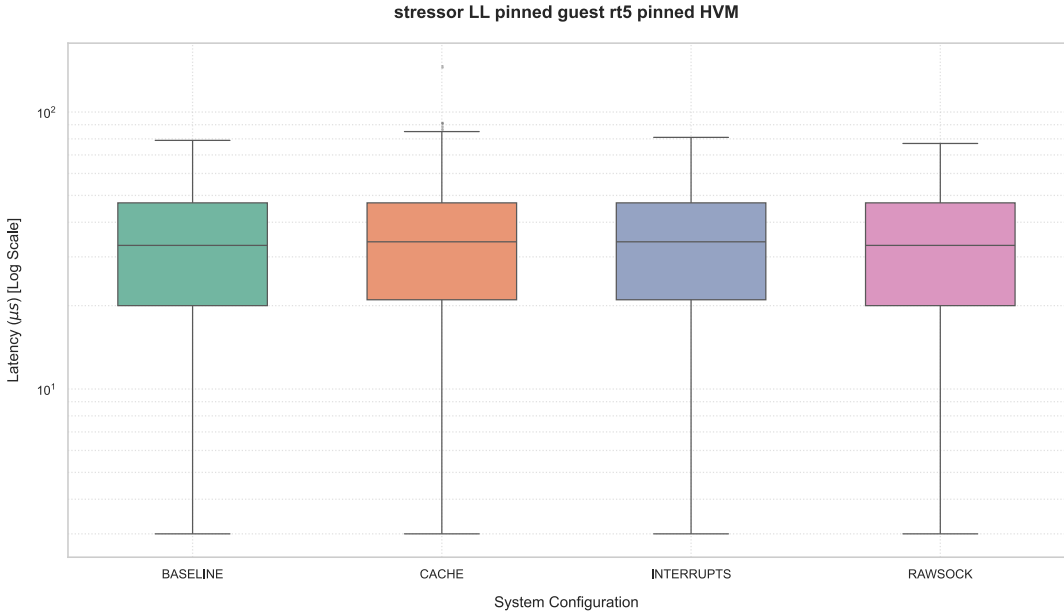
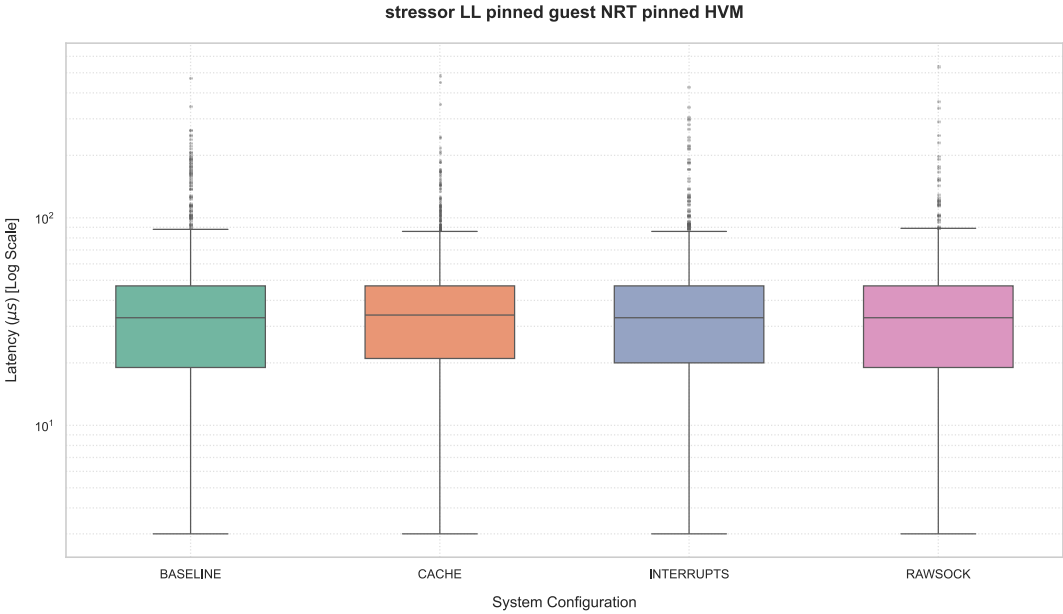
16 scenarios: unpinned/pinned × nrt/rt5 guest × 4 stressors

Pinning	Guest	Stressor	Avg (μs)	Max (μs)	Overflow	Note
No	nrt-hvm	baseline	35	392	0	
No	nrt-hvm	cache	36	61,611	88	Worst case overall
No	nrt-hvm	interrupts	35	61,826	12	
No	nrt-hvm	rawsock	35	58,344	11	
No	rt5-hvm	baseline	35	175	0	
No	rt5-hvm	cache	36	244	0	
No	rt5-hvm	interrupts	35	189	0	
No	rt5-hvm	rawsock	35	143	0	
Yes	nrt-pinned	baseline	32	450	0	
Yes	nrt-pinned	cache	33	414	0	
Yes	nrt-pinned	interrupts	32	4,225	47	Cluster overflow, end of run
Yes	nrt-pinned	rawsock	32	236	0	
Yes	rt5-pinned	baseline	32	72	0	
Yes	rt5-pinned	cache	34	99	0	
Yes	rt5-pinned	interrupts	33	80	0	
Yes	rt5-pinned	rawsock	33	357	0	

32-Scenario Stressor Study: LL Dom0



32-Scenario Stressor Study: LL Dom0 (Null Scheduler)



Full Results — Low-Latency (LL) Dom0 Kernel

16 scenarios: unpinned/pinned × nrt/rt5 guest × 4 stressors

Pinning	Guest	Stressor	Avg (μs)	Max (μs)	Overflow	Note
No	nrt-hvm	baseline	36	57,384	17	
No	nrt-hvm	cache	36	2,774	1	
No	nrt-hvm	interrupts	36	59,998	18	
No	nrt-hvm	rawsock	35	52,363	13	
No	rt5-hvm	baseline	36	192	0	
No	rt5-hvm	cache	36	2,772	19	Isolated anomaly
No	rt5-hvm	interrupts	35	144	0	
No	rt5-hvm	rawsock	35	162	0	
Yes	nrt-pinned	baseline	32	470	0	
Yes	nrt-pinned	cache	34	35,984	1	Single overflow, start of run
Yes	nrt-pinned	interrupts	32	423	0	
Yes	nrt-pinned	rawsock	32	536	0	
Yes	rt5-pinned	baseline	34	79	0	
Yes	rt5-pinned	cache	33	146	0	
Yes	rt5-pinned	interrupts	32	81	0	
Yes	rt5-pinned	rawsock	32	77	0	

Isolated Anomalies & the Cache Stressor

Three events that deviate from otherwise clean behaviour

Three Isolated Anomalies

- › **LL, rt5-hvm, cache:** 19 overflows, max 2.77 ms — while the same guest under every other stressor (including cache with the NRT kernel) stays at 0 overflows
- › **LL-pinned, nrt-pinned, cache:** a single overflow at 35.98 ms, occurring at cycle 913 of ~4.93M — right at the start of the run; the rest of the run is clean (secondary max ~536 μ s elsewhere)
- › **NRT-pinned, nrt-pinned, interrupts:** 47 overflows, but all clustered in the final ~3% of cycles — not distributed through the test



None of these three events has a counterpart in the same guest × stressor combination tested with the *other* kernel — suggesting single-run experimental noise or transient host interference rather than a reproducible kernel behaviour. With no repetitions in this design, the two hypotheses cannot be distinguished with certainty.

Why "Cache" Is the Wildcard

- › Produces the single worst result of the entire study (88 overflows, NRT kernel)...
- › ...yet is completely harmless in the same guest/ kernel combination once pinning is active (0 overflows)
- › Is the only stressor involved in **all three** isolated anomalies above
- › Compared to interrupts and rawsock — which impact the unpinned nrt-hvm guest more uniformly and predictably — cache shows the highest behavioural variance of any tested condition

88 → 0

OVERFLOWS: NO
PINNING →
PINNING (CACHE,
NRT)

3 / 3

ANOMALIES
INVOLVE THE
CACHE
STRESSOR

Final Considerations

- › **vCPU pinning influences latency quality more than any other factor** — almost zero overflows across all pinned configurations, regardless of Dom0 kernel or stressor
- › **The rt5-hvm (RT-patched) guest outperforms nrt-hvm in every tested condition**, pinning aside — RT guest patching is a robust, independent source of protection
- › **The unpinned nrt-hvm guest is unreliable under load** (except in the plain baseline scenario), with peaks up to ~62 ms regardless of the Dom0 kernel used
- › **The Low-Latency (LL) Dom0 kernel does not guarantee a systematic improvement** over the standard (NRT) kernel in this dataset — direct comparisons show crossed results depending on the specific guest/stressor combination
- › **The cache stressor generates the greatest variability** of any tested condition — including both the absolute worst case of the study and all three isolated anomalies



Takeaway: for Xen HVM guests under Dom0-side stress, *vCPU isolation* and *guest-side RT patching* are the two levers that reliably improve determinism — Dom0 kernel choice alone is not a substitute for either.

Noisy DomU Study: Mixed-Criticality Scenario Setup

A permanently running "noisy neighbour" VM — the most realistic mixed-criticality scenario

Experimental Design

Noisy DomU (always-on)

stress-ng: 16 CPU workers + 16 × 1 GB memory workers · no timeout · continuous load

Regular DomU (under measurement)

cyclicttest · 5 min · FIFO-99 · 50 μs interval · histogram 1000 μs

Dom0 (4 vCPUs)

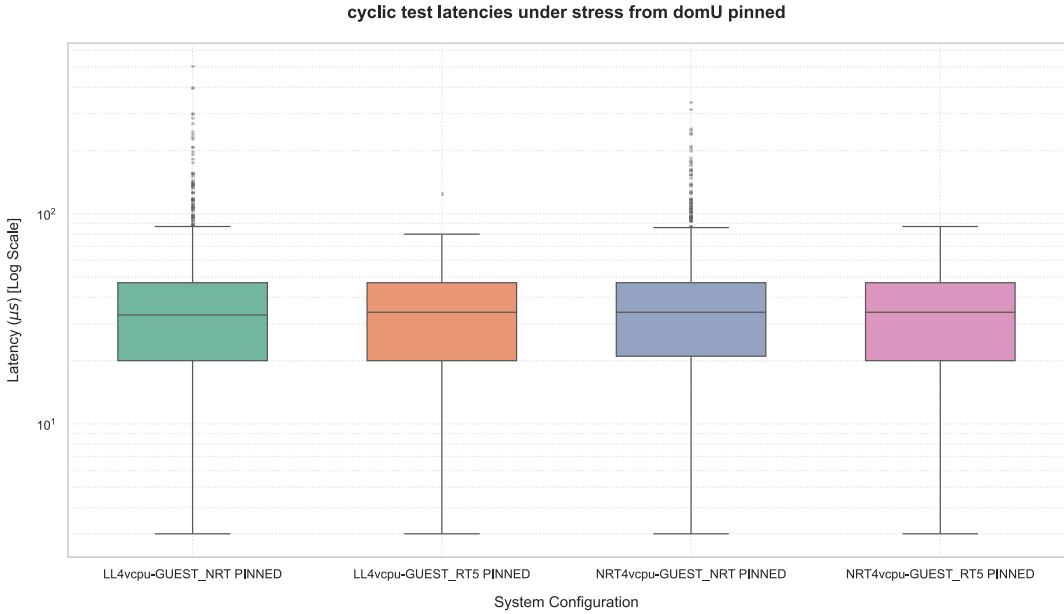
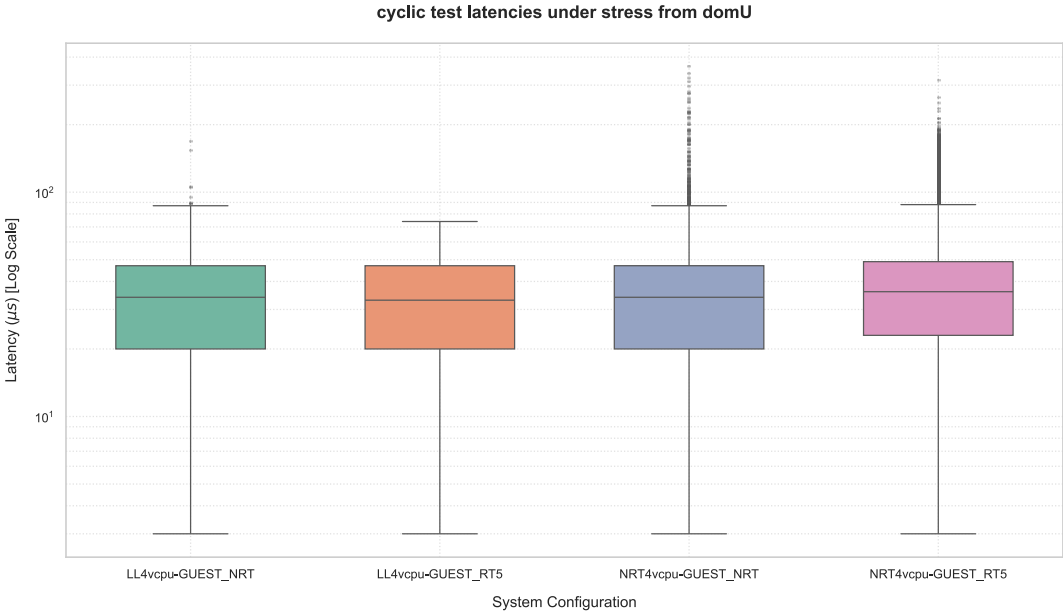
Control domain — no additional stress from Dom0 itself

Orchestrated end-to-end by a bash script: reboots Dom0 onto the target kernel, launches the noisy DomU, then runs each regular DomU in turn — clearing Dom0 caches between runs. Each combination executed once (no repetitions).

8 Test Configurations

Dom0 Kernel	Pinning	DomU Kernel
NRT-4vcpu	No	nrt-hvm (non-RT)
	No	rt5-hvm (RT)
	Yes	nrt-pinned-hvm
	Yes	rt5-pinned-hvm
LL-4vcpu (rt5-ll)	No	nrt-hvm (non-RT)
	No	rt5-hvm (RT)
	Yes	nrt-pinned-hvm
	Yes	rt5-pinned-hvm

Noisy DomU: Results



Noisy DomU: Results

Results — 8 Configurations

Dom0	Pinning	DomU	Avg	WCET (μs)	Overflows
NRT	No	nrt-hvm	33 μs	62,566 ⚠	4
NRT	No	rt5-hvm	36 μs	312	0
NRT	Yes	nrt-hvm	33 μs	336	0
NRT	Yes	rt5-hvm	33 μs	87	0
LL	No	nrt-hvm	32 μs	167	0
LL	No	rt5-hvm	32 μs	74	0
LL	Yes	nrt-hvm	33 μs	502	0
LL	Yes	rt5-hvm	33 μs	123	0

Isolating the Pinning & DomU-Kernel Effects

Minimum latency: 3 µs across all 8 tests

A stable and consistent value regardless of kernel, pinning, or DomU configuration — indicating that the "idle" path (no interference) is not influenced by the noisy neighbour. Only the *tail* of the distribution is affected.



This is the closest approximation to a **real mixed-criticality deployment**: a high-criticality guest running alongside a permanently active, resource-intensive low-criticality VM.

vCPU Pinning: Opposite Effects Depending on Dom0 Kernel

- On NRT Dom0, pinning is **transformative**: it eliminates the catastrophic outlier entirely — 62,566→336 µs (non-RT DomU), 312→87 µs (RT DomU) — and zeroes out all overflows
- On LL Dom0, pinning is **mildly counter-productive**: it slightly *increases* the maximum — 167→502 µs (non-RT DomU), 74→123 µs (RT DomU) — though both remain orders of magnitude below the NRT critical threshold



Pinning is a powerful countermeasure **only in the absence** of a low-latency Dom0 kernel. Once Dom0 is already tuned, pinning adds no further benefit and may add slight overhead by concentrating interference onto a dedicated core.

RT DomU Kernel: Consistent Benefit in All 4 Dom0 Configs

Dom0 Config	non-RT DomU	RT DomU	Reduction
NRT-4vcpu	62,566 µs	312 µs	−99.5%
NRT-4vcpu-pinned	336 µs	87 µs	−74%
LL-4vcpu	167 µs	74 µs	−56%
LL-4vcpu-pinned	502 µs	123 µs	−75%



Overlapping benefits: any LL-4vcpu configuration, pinned or not, stays within a few hundred µs regardless of DomU kernel. It's the total absence of *both* protections that produces the campaign's sole out-of-scale result.

Noisy DomU: Conclusions

- **1 critical case out of 8:** NRT + no pinning + non-RT DomU → 62.5 ms WCET, 4 overflows (at cycles 8,323 / 20,698 / 24,840 / 37,984). The total absence of both protections is the only truly dangerous combination.

Conclusions

- › **7 out of 8 configurations stay under 1 ms** WCET — the noisy neighbour risk is manageable with modest mitigations
- › **vCPU pinning** (on NRT Dom0): eliminates 62.5 ms → 336 μ s (nrt DomU) and 312 → 87 μ s (rt5 DomU)
- › **LL kernel on Dom0** inherently guarantees stable behaviour regardless of pinning
- › **RT DomU** systematically reduces tail latency vs non-RT twin under identical Dom0 conditions
- › On LL Dom0: pinning slightly *increases* WCET (167→502 μ s for nrt) — concentrated interference on a dedicated core

Xen TACLe: Configuration & Scenario Design

Run on LL Dom0 + RT DomU (Xen's optimal configuration) — 5 benchmarks × 4 scenarios

matrix1
KERNEL
Raw compute. Cache-bound. 1M loops.

huff_enc
SEQUENTIAL
Data compression. 1M loops. ~16 µs avg.

lift
APPLICATION
Elevator controller. 1M loops. ~19 µs avg.

test3
STRESS TEST
WCET stress. 10k loops. ~8,300 µs avg.

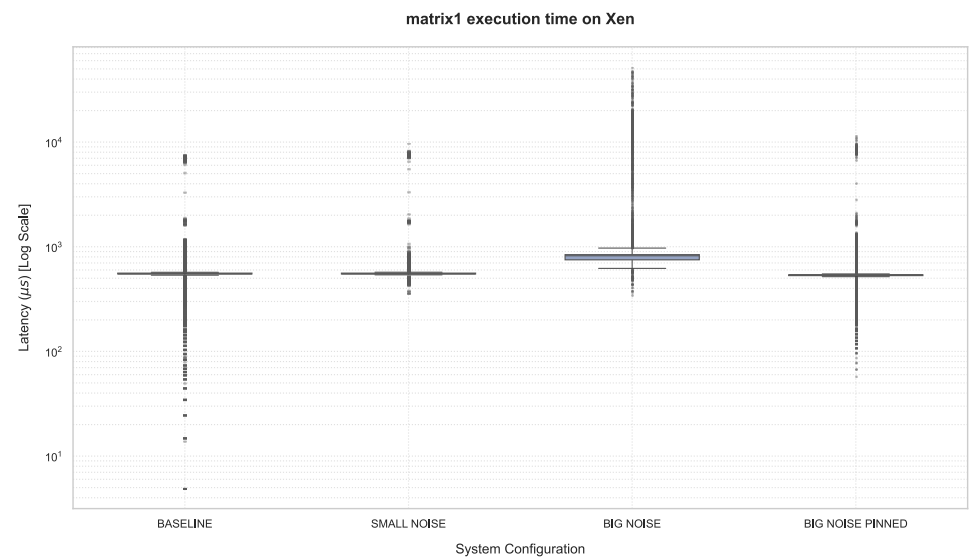
DEBIE
PARALLEL
Aerospace multi-task. 10k loops. ~27,000 µs avg.

Test Scenarios (Xen: LL Dom0 + RT DomU · 4 dedicated Dom0 vCPUs)

Scenario	Noisy DomU	Pinning	Purpose
Baseline	None	No	Reference WCET without interference
Small Noise	2 vCPUs, 4 GB RAM	No	Light inter-guest interference
Big Noise	16 vCPUs, 16 GB RAM	No	Heavy inter-guest interference (stress-ng CPU+VM)
Big Noise Pinned	16 vCPUs, 16 GB RAM	Yes	Effectiveness of static vCPU allocation under heavy load

 **Parallel structure with KVM TACLe:** Same 5 benchmarks, same 4 scenarios. KVM used RT-RT (KVM's best); Xen uses LL-RT (Xen's best). Direct comparison is meaningful.

Xen TACLe: matrix1 — Raw Compute (Kernel Benchmark)



matrix1: Execution Time Across Scenarios

Scenario	Min (μs)	Avg (μs)	WCET (μs)	Jitter (μs)
Baseline	0	0 ± 0.17	9	9
Small Noise	0	0 ± 0.13	8	8
Big Noise	0	0.01 ± 0.3	45	45
Big Noise Pinned	0	0.01 ± 0.2	11	11

Xen TACLe: matrix1 — Analysis

Raw compute (kernel benchmark): sensitivity to noise

Analysis

- › Average latency reads 0 μ s in every scenario — the task is entirely cache-resident, essentially invisible to the scheduler except at its absolute tail
- › Baseline and Small Noise are statistically indistinguishable (9 vs 8 μ s WCET) — light interference has no measurable effect
- › Big Noise is the only scenario that perturbs the task, driving WCET to 45 μ s (**5 $\times$ baseline**)
- › Pinning recovers to 11 μ s, just 2 μ s above baseline — near-complete mitigation
- › Same qualitative pattern as KVM (1 $\rightarrow$ 6 μ s, 6 $\times$), at a higher absolute scale — consistent with Xen's baseline hypervisor overhead being higher than KVM's for this ultra-lightweight kernel

✓ matrix1 confirms the same story on Xen as on KVM: isolated, cache-resident kernel loops are nearly immune to virtualization noise, and pinning almost completely erases the one disturbance that does appear.

5 $\times$

WCET INCREASE (BIG NOISE)

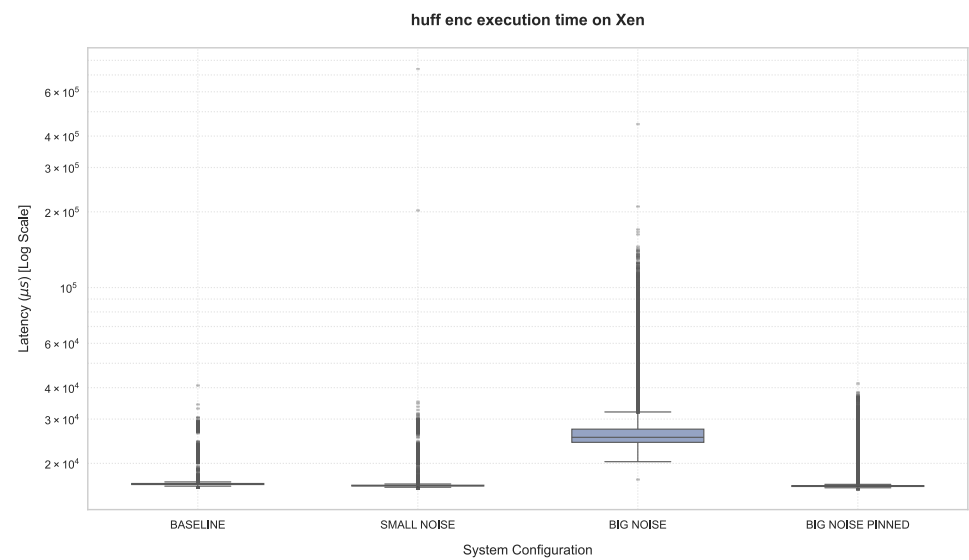
+22%

RESIDUAL WCET VS BASELINE (AFTER PINNING)

-76%

WCET REDUCTION (PINNING)

Xen TACLe: huff_enc — Data Compression (Sequential)



huff_enc: Execution Time Across Scenarios

Scenario	Min (μs)	Avg (μs)	WCET (μs)	Jitter (μs)
Baseline	15	16 ± 1	35	20
Small Noise	15	16 ± 1	35	20
Big Noise	15	19 ± 6	182	167
Big Noise Pinned	15	16 ± 2	42	27

Xen TACLe: huff_enc — Analysis

Data compression (sequential): sensitivity to noise

Analysis

- › Baseline and Small Noise are statistically identical (35 μ s WCET, 20 μ s jitter in both) — huff_enc is completely immune to a light noisy neighbor (2 vCPU/4GB)
- › Big Noise is the only scenario that meaningfully disturbs the task: WCET more than quintuples (35→182 μ s, **+420%**), jitter grows 8× (20→167 μ s)
- › Pinning restores near-baseline behavior (42 μ s WCET, 27 μ s jitter) — a **-77%** reduction from Big Noise
- › Compared to KVM (baseline WCET 64 μ s), Xen's baseline (35 μ s) is noticeably tighter — LL-RT appears to give huff_enc a lower structural floor than KVM's RT-RT

✓ huff_enc on Xen is essentially immune to light interference and only meaningfully disturbed by heavy noisy-neighbor load — pinning nearly fully reverses that disturbance.

+420%

WCET INCREASE (BIG NOISE)

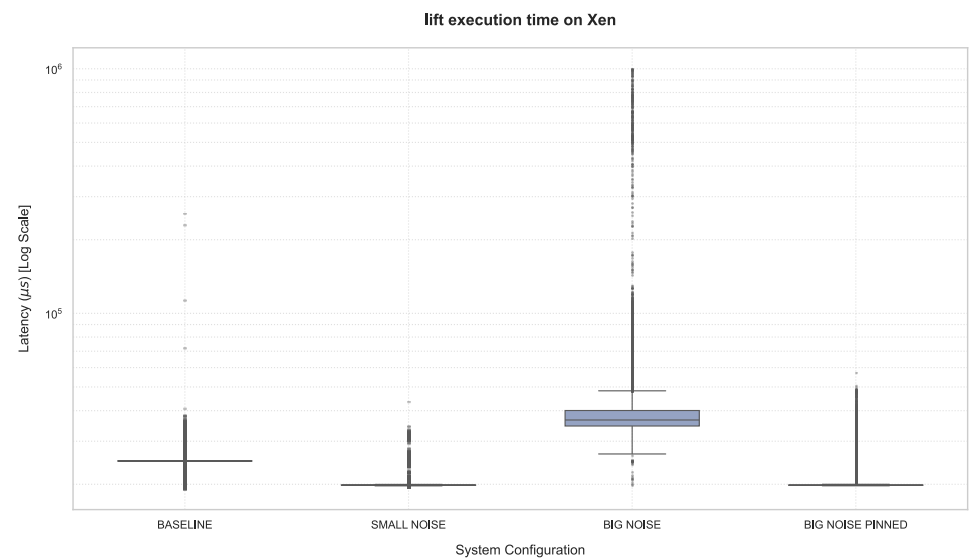
8×

JITTER GROWTH (BIG NOISE)

-77%

WCET REDUCTION (PINNING)

Xen TACLe: lift — Elevator Controller (Application)



lift: Execution Time Across Scenarios

Scenario	Min (μs)	Avg (μs)	WCET (μs)	Jitter (μs)
Baseline	19	19 ± 1	85	66
Small Noise	19	19 ± 1	36	17
Big Noise	17	20 ± 5	142	125
Big Noise Pinned	19	19 ± 2	43	24

Xen TACLe: lift — Analysis

Elevator controller (application): sensitivity to noise

Analysis

- › Xen's baseline (85 μ s WCET) actually carries more jitter than its own Small Noise run (36 μ s) — a plausible single-run artifact (no repetitions), not evidence that noise improves determinism
- › Big Noise pushes WCET to 142 μ s (**+67%** vs baseline), the worst point in the series
- › Pinning fully recovers determinism: 43 μ s under Big Noise+Pinning (**−70%**), close to the Small-Noise figure
- › **Vs. KVM:** lift's Big-Noise WCET on KVM spiked to 280 μ s (4.6 $\times$ baseline); on Xen it reaches only 142 μ s (1.7 $\times$) — **Xen is roughly 2 $\times$ more resilient** on this control-loop workload



Unlike on KVM, where lift suffered a dramatic 280 μ s spike under heavy noise, Xen keeps lift's worst case within 142 μ s — under half of KVM's degradation — and pinning brings it back to near-baseline levels.

+67%

WCET INCREASE (BIG NOISE)

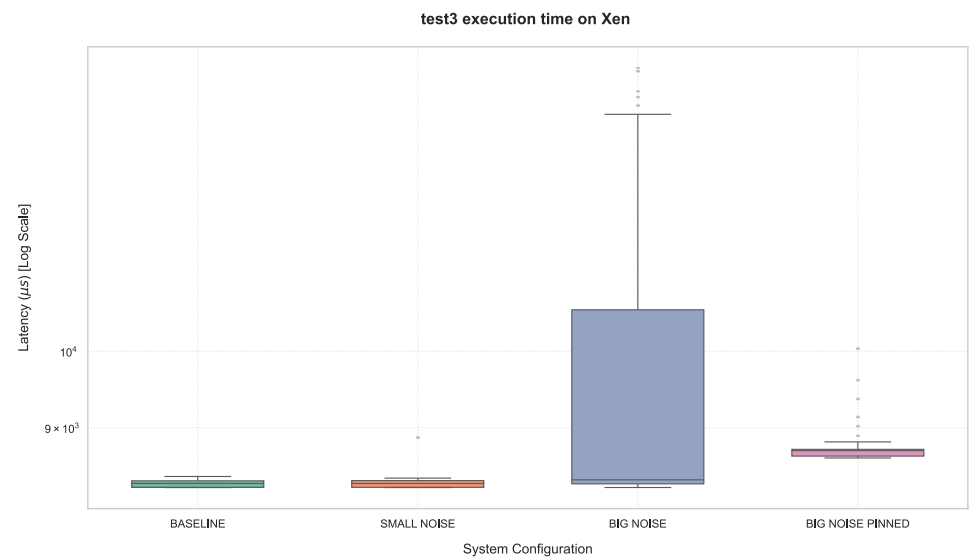
+19%

RESIDUAL WCET VS SMALL NOISE (AFTER
PINNING)

−70%

WCET REDUCTION (PINNING)

Xen TACLe: test3 — WCET Stress Test



test3: Execution Time Across Scenarios

Scenario	Min (μs)	Avg (μs)	WCET (μs)	Jitter (μs)
Baseline	8,286	8,328 ± 38	8,414	128
Small Noise	8,286	8,331 ± 40	8,885	599
Big Noise	8,287	9,183 ± 1317	14,786	6,499
Big Noise Pinned	8632	8,701 ± 44	10,037	1,405

Xen TACLe: test3 — Analysis

WCET stress test: sensitivity to noise

Analysis

- › Xen's baseline jitter (128 μ s) is the tightest of any condition in this series — a well-behaved, medium-weight loop when undisturbed
- › Small Noise nearly quintuples the jitter (128→599 μ s) despite barely moving the average — even a light neighbor measurably lengthens the tail for this longer-running task
- › Big Noise is far more severe: WCET reaches 14,786 μ s (**+76%**), jitter balloons to 6,499 μ s (**~51× baseline**) — the worst relative degradation of any TACLe benchmark on Xen
- › Pinning recovers most, but not all: WCET drops to 10,037 μ s (**−32%** from Big Noise), still well above baseline — unlike the near-full recovery seen in lighter benchmarks
- › Unlike KVM's test3 anomaly (where pinning made things worse), Xen's pinning goes in the expected direction — but the recovery is partial, not complete



test3 is the least resilient TACLe benchmark to Dom0 noise on Xen: even after pinning, WCET remains ~1,600 μ s above baseline — a residual footprint pinning alone cannot fully eliminate for this longer-running compute loop.

51×

JITTER EXPLOSION (BIG NOISE VS BASELINE)

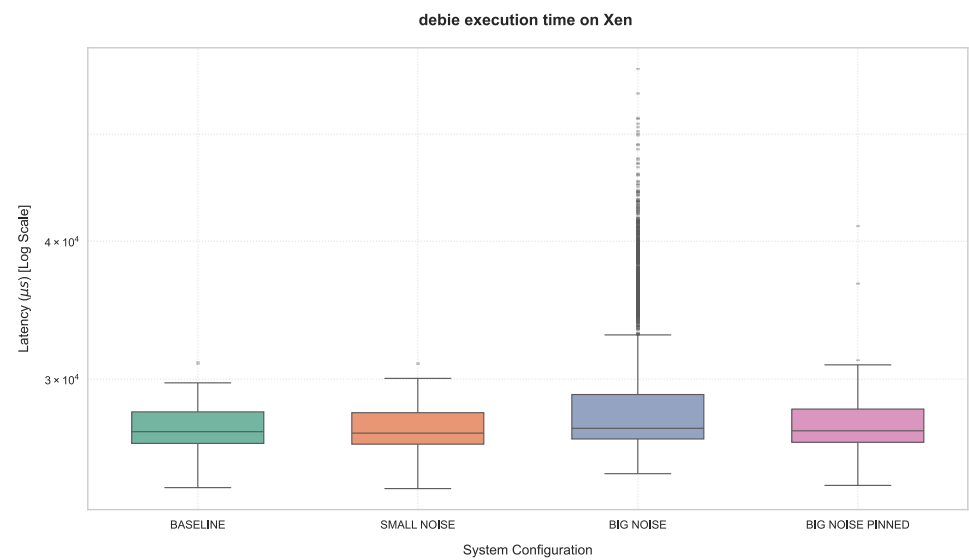
+76%

WCET INCREASE (BIG NOISE)

−32%

WCET REDUCTION (PINNING) — PARTIAL ONLY

Xen TACLe: DEBIE — Aerospace Multi-Task Parallel Benchmark



DEBIE: Execution Latency Across Scenarios

Scenario	Min (μs)	Avg ± SD (μs)	WCET (μs)	Jitter (μs)
Baseline	23,921	27,176 ± 1,309	31,048	7,127
Small Noise	23,873	27,118 ± 1312	31,022	7,149
Big Noise	24,627	28,197 ± 3,278	57,314	32,687
Big Noise Pinned	24,034	27,274 ± 1328	41,266	17,232

Xen TACLe: DEBIE — Analysis

Multi-task parallel workload: sensitivity to noise

Analysis

- › Noise expands the tail massively (+85% WCET) while barely touching the average (+4%) — same pattern as KVM
- › Small noise has virtually *no effect* (31,022 vs 31,048 μ s) — DEBIE's inherent variance absorbs it
- › Pinning reduces WCET (28%) but cannot eliminate the multi-task synchronisation overhead
- › Xen baseline WCET (31,048 μ s) is slightly better than KVM (32,409 μ s) on this parallel workload



DEBIE's variance is fundamentally driven by its multi-task inter-task coordination, not purely by hypervisor scheduling. Pinning alone is insufficient — task-level isolation strategies are needed for such workloads.

+85%

WCET INCREASE (BIG NOISE)

+4%

AVERAGE INCREASE (BIG NOISE)

-28%

WCET WITH PINNING

Xen TACLe: Summary & Pinning Trade-offs under Dom0 Stress

Max Latency Summary (µs) — Noisy DomU scenarios

Benchmark	Baseline	Small Noise	Big Noise	Big Noise Pinned	Pinning Effect
DEBIE	31,048	31,022	57,314	41,266	-28%
huff_enc	35	35	182	42	-77%
lift	85	36	142	43	-70%
matrix1	9	8	45	11	-76%
test3	8,414	8,885	14,786	10,037	-32%

Dom0 Stress Study — Pinning Paradox

Under Dom0 stress, pinning systematically reduces mean & p99 but can worsen WCET:

Benchmark	Δ Mean	Δ p99	Δ Max (WCET)
debie	-26.5%	-35.8%	-45.8%
huff_enc	-34.1%	-31.4%	+142.6% ⚠
lift	-46.3%	-37.0%	+326.5% ⚠
matrix1	-30.7%	-26.6%	-72.1%
test3	-15.4%	-37.2%	-8.2%

Interpreting the Paradox

- Pinning reduces **variance in the common case** (lower mean/p99 across all 5 benchmarks)
- But if the pinned core is hit by a housekeeping IRQ or Dom0 timer, the interference is **concentrated on one core** instead of distributed
- Result: rare but larger isolated WCET spikes for lift and huff_enc
- Full core isolation (IRQ affinity on all interrupts) would be needed to prevent this

PV & PREEMPT_RT: An Incompatibility, Not a Priority Bug

With the HVM priority-inversion fix confirmed, we extend to guest types with no Device Model at all — PV and PVH

What Happened

- › Configuring a PV guest with a PREEMPT_RT kernel reproduced the same failure seen earlier during initial setup — this time, with terminal-routed logs, the cause was identifiable
- › Logs revealed a **soft CPU lockup**, pointing directly at the PREEMPT_RT patch itself as the source of the failure
- › **Hypothesis:** PREEMPT_RT replaces spinlocks with mutexes at a low level — a change that likely conflicts with how paravirtualization cooperates with the hypervisor
- › This also explains why the RT patch fails on **Dom0 itself**: Dom0 is inherently a PV guest

A Second Dead End: PVH Dom0

An attempt to configure Dom0 itself as a PVH guest resulted in **continuous automatic reboots**. With no system logs or graphical output available to diagnose the failure, the root cause could not be identified — this configuration was abandoned.



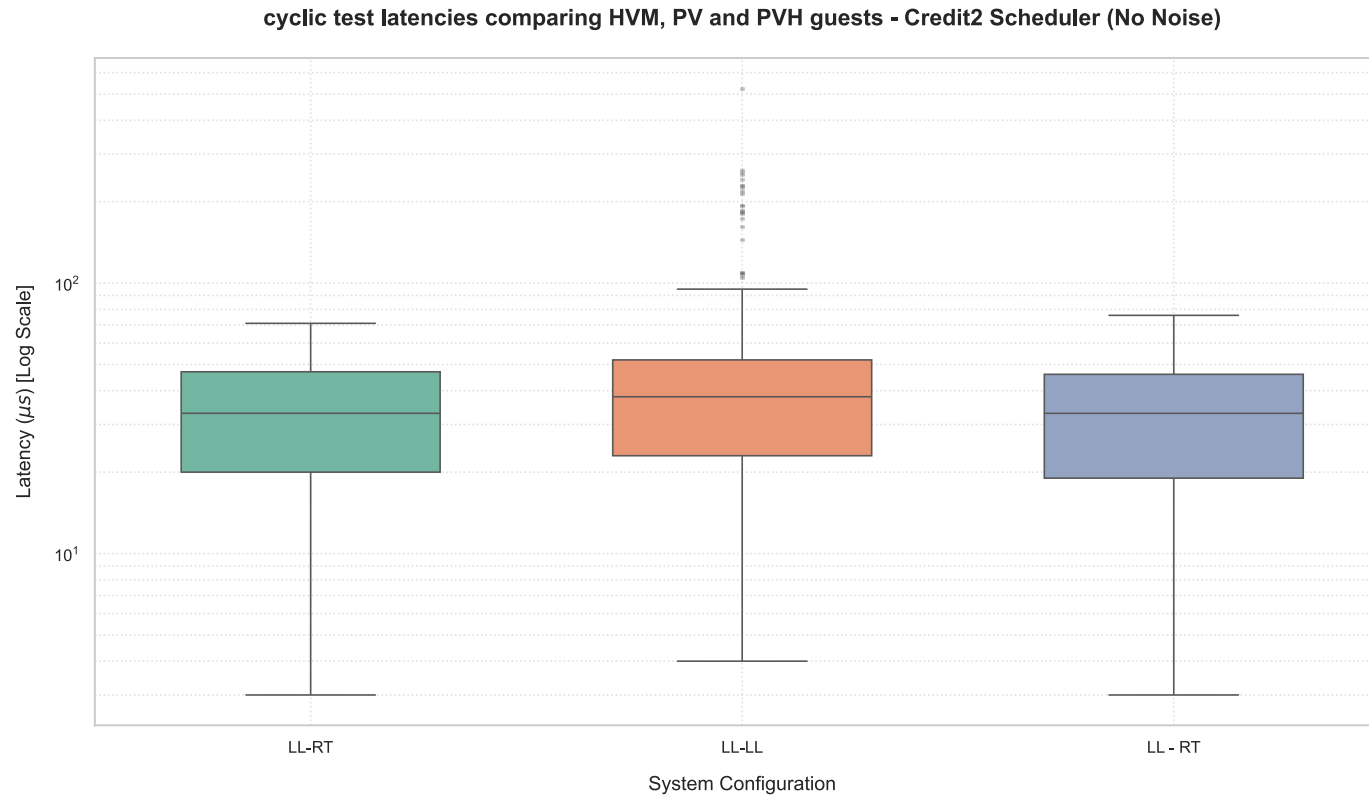
Consequence for methodology: since PREEMPT_RT cannot run on a PV guest, all subsequent PV tests use the **Low-Latency (LL) kernel** on the DomU in place of the RT kernel — PVH remains the only guest type where RT is tested.



PV is therefore structurally excluded from hard real-time use on Xen — not due to a fixable scheduling bug, but a kernel-patch incompatibility at the virtualization layer.

Baseline: LL Dom0, No Stress, No Pinning

PV (LL DomU, forced) vs PVH (RT DomU) — dynamic Credit2 scheduler



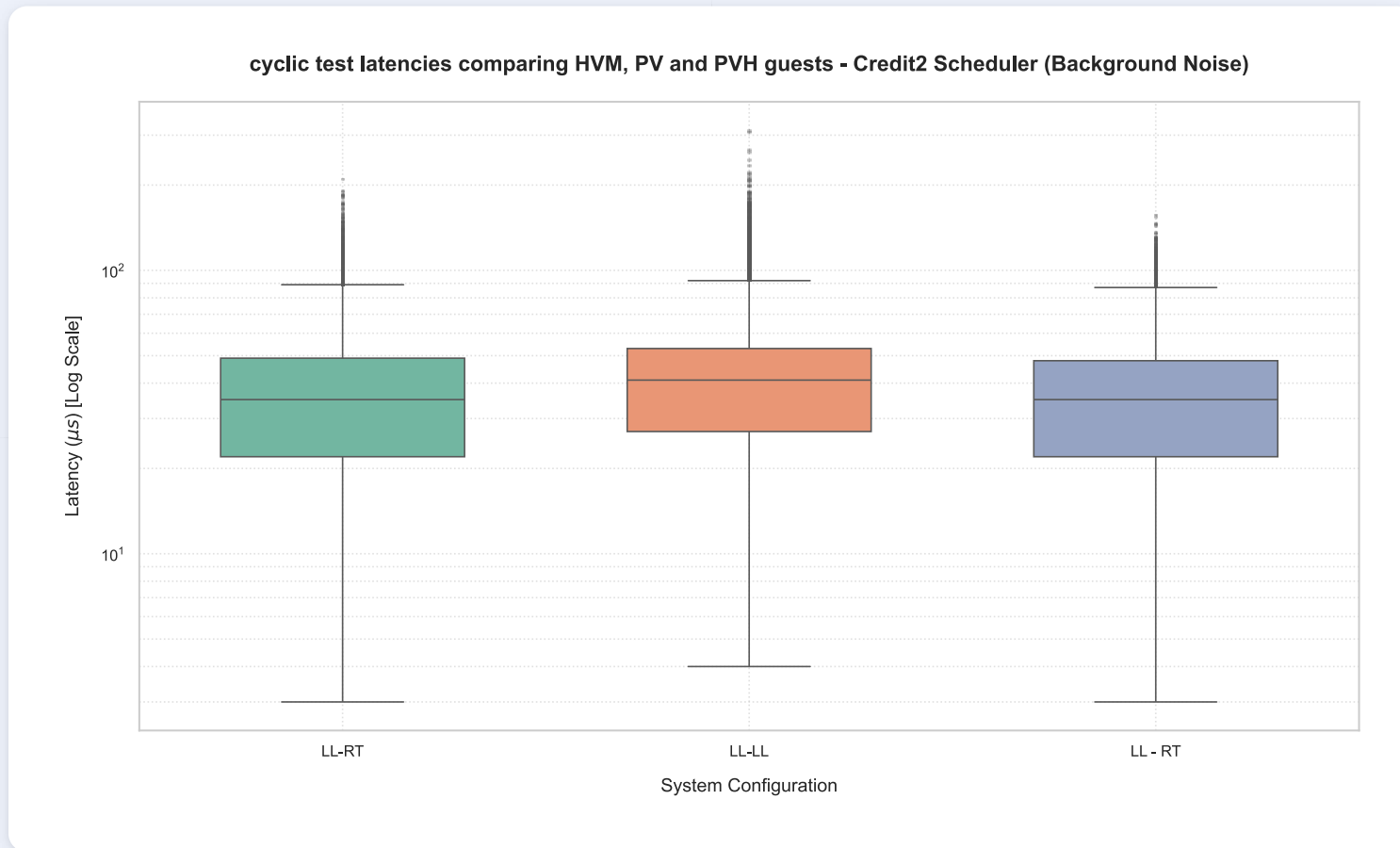
Reading the Gap

- PV's higher average (37 μ s, vs 32 μ s for PVH) reflects the baseline overhead of a software-only virtualization path with no hardware-assisted extensions
- The WCET gap is far larger than the average gap — this is a **tail** effect: PV, forced onto the LL kernel, has no RT scheduling guarantees to bound its worst case
- PVH's zero overflows and 76 μ s WCET, achieved *without pinning*, show that a genuine RT guest kernel on a lean, DM-free architecture is enough for strong determinism even under dynamic Credit2 scheduling

✓ This baseline gap is the direct, measurable consequence of the PV/ PREEMPT_RT incompatibility described on the previous slide — not a scheduling artifact.

PV vs PVH vs HVM Under Dom0 Stress

Credit2 scheduler — background stress workload in Dom0, `cyclicttest` probe in PV/PVH/HVM guests



Key Observations

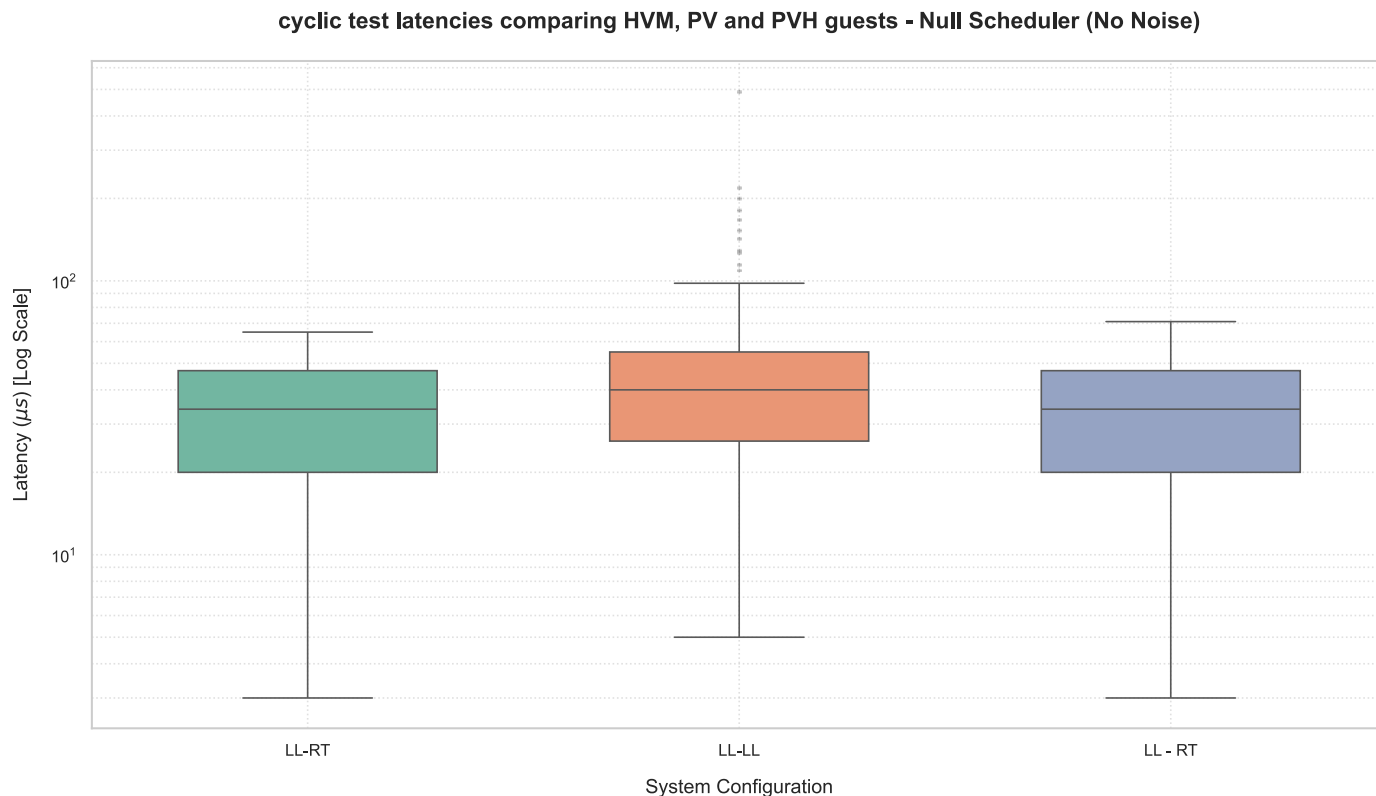
i Removing the Device Model isolates PV/PVH behaviour from QEMU-induced preemption — but PV pays for it: **PREEMPT_RT incompatibility** forces a Low Latency kernel, moving the bottleneck from the hypervisor to guest-level scheduling.

✓ **PVH ≈ HVM** under contention — both stay within predictable bounds, confirming PVH as a lightweight alternative with full RT resilience.

✗ **PV is the worst performer** of the three technologies: the lack of RT guest optimizations outweighs any benefit from its lighter virtualization footprint.

PV vs PVH vs HVM Under Static Allocation: Baseline

Strict vCPU-to-pCPU pinning on Dom0 and DomU — emulating an offline NULL scheduler, unstressed conditions



Key Observations

✓ **PVH baseline (71 μs) ≈ HVM (65 μs)** even without stress — static isolation confirms the architectural nuance is minor between the two.

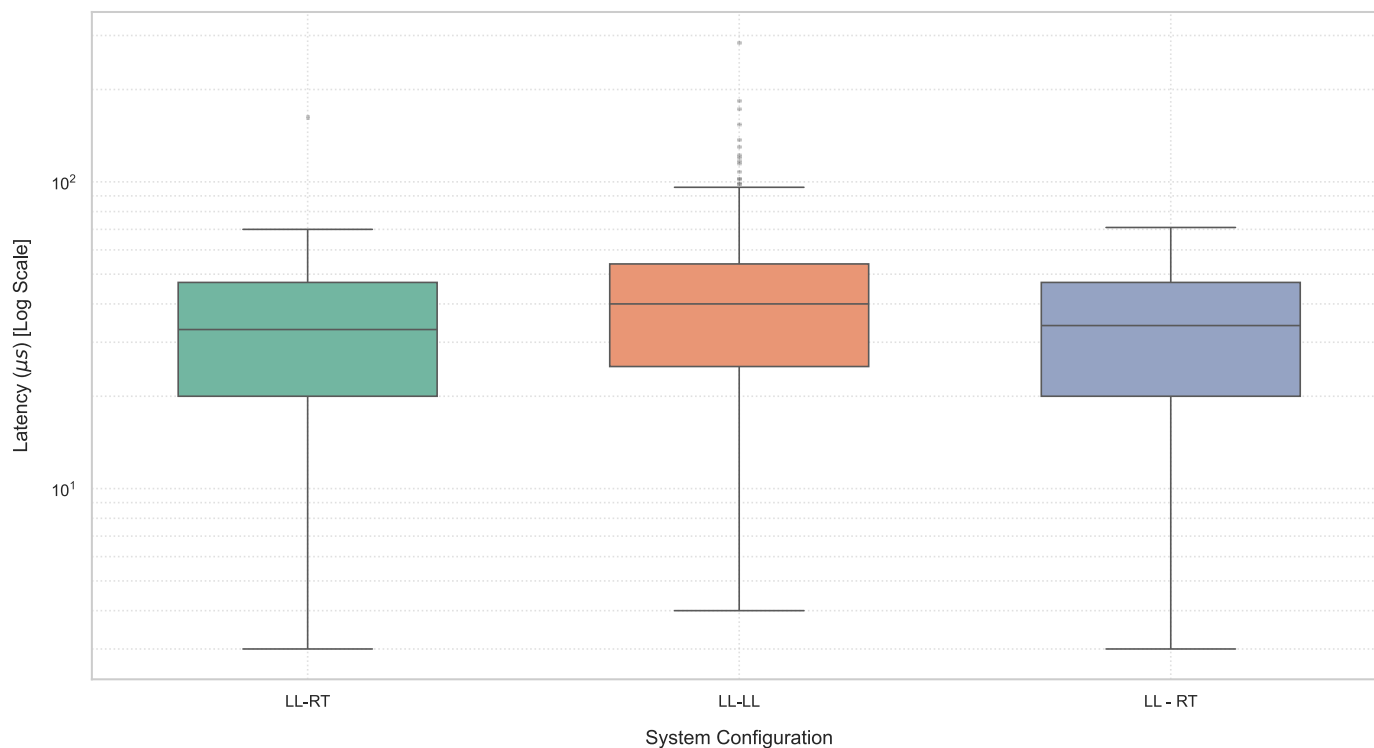
✗ **Pinning alone cannot fix PV:** even under ideal, unstressed hardware isolation, PV's WCET (492 μs) remains ~7× worse than HVM/PVH — the penalty comes from the PREEMPT_RT-incompatible Low Latency kernel, not from host contention.

i This clean baseline isolates the *inherent* virtualization overhead — any degradation observed once Dom0 stress is introduced can be attributed specifically to contention, not to scheduling variability.

PV vs PVH vs HVM Under Static Allocation + Dom0 Stress

Severe `stressdom0` background workload in the privileged domain, static vCPU pinning retained on both PV and PVH DomUs

cyclic test latencies comparing HVM, PV and PVH guests - Null Scheduler (Background Noise)



Key Observations

★ **PVH + Pinning + PREEMPT_RT under stress: 71 μs WCET** — *identical* to its own unstressed baseline. Static isolation successfully shields PVH's critical execution paths, achieving perfect WCET stability regardless of host load.

✗ **PV remains susceptible to host contention** even with static pinning: WCET degrades from 492 μs to a still-measurable interference pattern, confirming that hardware isolation alone cannot compensate for the PREEMPT_RT-incompatible Low Latency kernel.

i PVH's isolation from the Device Model plus RT-kernel compatibility lets it fully absorb Dom0 saturation, while PV's guest-level scheduling limitations remain exposed regardless of pinning strategy.

PV vs PVH vs HVM: Summary

PV (Paravirtualization)

- > No Device Model — no QEMU in Dom0
- > **PREEMPT_RT incompatible** — forced fallback to Low Latency kernel
- > Bottleneck shifts from hypervisor to guest-level scheduling
- > Lighter footprint cannot compensate for missing RT optimizations

HVM (Full Virtualization)

- > QEMU Device Model in Dom0
- > PREEMPT_RT compatible
- > Historical priority-inversion bug: confirmed FIXED
- > Manages latency bounds well under contention

PVH

- > No Device Model, no QEMU dependency
- > PREEMPT_RT compatible
- > Isolated from Device Model interference by design
- > Resilience to contention on par with HVM

Baseline WCET Comparison

Configuration	DomU Kernel	Min	Avg ± SD	WCET	Overflows
PV	Low-Latency	4 µs	37 ± 15 µs	525 µs	0
PVH	Real-Time (PREEMPT_RT)	3 µs	32 ± 15 µs	76 µs	0

PV vs PVH vs HVM: Summary

Worst-Case Latencies Under Stress

Config	Stress Average	Stress WCET	Average Δ%	WCET Δ%
PV	40 ± 16 μs	312 μs	+8.16%	-40.57%
PVH	36 ± 15.5 μs	157 μs	+9.82%	+106.58%

Baseline Pinned WCET Comparison

Configuration	DomU Kernel	Min	Avg ± SD	WCET	Overflows
PV	Low-Latency	4 μs	40 ± 15 μs	492 μs	0
PVH	Real-Time (PREEMPT_RT)	3 μs	33 ± 15 μs	71 μs	0

Worst-Case Latencies Under Stress (Pinned)

Config	Stress Average	Stress WCET	Average Δ%	WCET Δ%
PV	39 ± 15 μs	285 μs	-2.13%	-42%
PVH	33 ± 15 μs	71 μs	+0.14%	0%

05

Conclusions & Takeaways

What did we learn, and what does it mean for real-time
hypervisor design?

CONCLUSIONS

KVM vs Xen: Head-to-Head Summary

Metric	KVM (RT-RT)	Xen (LL-RT + Pinning)
Baseline WCET	81 μs	65 μs (HVM) / 71 μs (PVH)
Under Stress WCET	~9,800 μs (283 overflows)	163 μs (HVM) / 71 μs (PVH pinned)
Stress + Isolation	~1,935 μs (14 overflows)	71 μs, 0 overflows (PVH)
Scheduling Overhead	~8 μs avg (hypervisor-transparent)	~32-35 μs avg (structural Xen constant)
TACLe (lift, pinned)	67 μs	43 μs
Hard RT Feasibility	Borderline — stress cannot be fully contained	Yes — with LL/RT + vCPU pinning + PVH
Priority Inversion	N/A	Fixed in Xen 4.17.3



KVM advantage: Much lower average latency overhead (~8 μs vs ~32 μs). Better for soft-RT applications where average matters more than WCET.



Xen advantage: Dramatically better WCET containment under stress. PVH + pinning achieves provably stable bounds even with heavy colocated workloads.

CONCLUSIONS

Key Takeaways

- ① **Determinism is an end-to-end property.** Optimizing only the host OR the guest is insufficient — and can make things worse. The NRT-RT anomaly (6.1 ms WCET!) is a concrete demonstration.
- ② **vCPU pinning is the dominant factor for Xen.** It reduces overflows from 179 to 48 across the 32-scenario stressor sweep — more impactful than the Dom0 kernel choice. But it is not universally beneficial for WCET under concentrated interference.
- ③ **The Xen QEMU priority-inversion bug is fixed in Xen 4.17.3.** Elevating QEMU to SCHED_FIFO priority 99 yields no improvement. Modern HVM decouples timer/interrupt delivery from the Device Model process.
- ④ **PVH + PREEMPT_RT + Pinning is Xen's optimal configuration.** 71 μ s WCET even under Dom0 stress — *identical* to its own unstressed baseline. Zero overflows. The PVH architecture removes the Device Model bottleneck while supporting full RT patches.
- ⑤ **KVM cannot fully contain hardware-interrupt-induced latency spikes** under heavy colocated load — even with CPU isolation (14 overflows, 1.9 ms WCET remain). Xen's Type-1 architecture provides fundamentally better temporal isolation for hard-RT workloads.

Open Questions & Future Work

Memory Bandwidth Partitioning

Intel RDT / Memguard could further constrain cache-induced latency spikes (cache stressor was most dangerous). Partially explored but needs deeper evaluation.

Statistical WCET Estimation

5-minute runs may miss rare events. Extreme Value Theory (EVT) / pWCET estimation would strengthen the analysis with probabilistic bounds.

ARM / RISC-V Platforms

x86 VT-x behaviour differs from ARM interrupt handling. How do these findings translate to embedded platforms (RPi, Jetson)?

Unikraft Unikernels

Unikraft unikernels as alternative guests — minimal scheduler, no kernel bloat. Promising for extremely lightweight real-time guests but out of scope for final analysis.

Acknowledged Limitations

- › Single physical machine — hardware-specific results (AMD CPU, MSI board)
- › Single run per configuration — no statistical repetition for TACLe and stressor studies
- › NULL scheduler unavailable in Xen 4.17.3 — vCPU pinning used as approximation
- › PV analysis limited by PREEMPT_RT incompatibility — LL kernel used as fallback
- › Unikraft comparison not completed for final submission

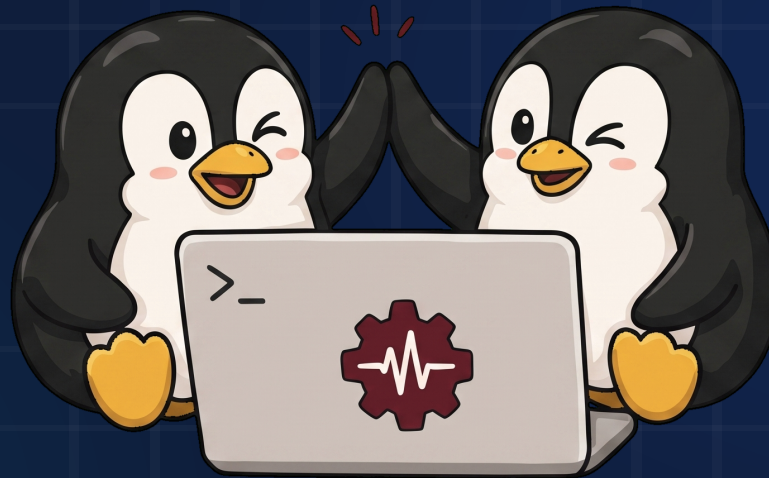


Good research raises more questions than it answers. These limitations are opportunities for future investigation — not fundamental flaws in the methodology.

References

- **L. Abeni, D. Faggioli**, *"An Experimental Analysis of the Xen and KVM Latencies,"* IEEE International Symposium on Real-Time Computing (ISORC), 2019. DOI: 10.1109/ISORC.2019.00014
- **L. Abeni, D. Faggioli**, *"Using Xen and KVM as real-time hypervisors,"* Journal of Systems Architecture, vol. 106, 2020. DOI: 10.1016/j.sysarc.2020.101709
- **M. Cinque, L. De Simone, D. Ottaviano**, *"Temporal isolation assessment in virtualized safety-critical mixed-criticality systems: A case study on Xen hypervisor,"* The Journal of Systems and Software, vol. 216, 112147, 2024. DOI: 10.1016/j.jss.2024.112147
- **L. Abeni**, *"Virtualized real-time workloads in containers and virtual machines,"* Journal of Systems Architecture, vol. 154, 103238, 2024. DOI: 10.1016/j.sysarc.2024.103238
- **S. Kuenzer, V.-A. Bădoiu, H. Lefeuvre et al.**, *"Unikraft: Fast, Specialized Unikernels the Easy Way,"* Proceedings of EuroSys '21, 2021.
- **TACLeBench Suite v1.9** — Benchmark programs used: matrix1, huff_enc, lift, test3, DEBIE. Available at: <https://github.com/tacle/tacle-bench>

THANK YOU
FOR YOUR ATTENTION.
QUESTIONS?



AUTHORS

Rita Marino · Matteo Arnese

REPOSITORY

github.com/Rita5351/Xen-vs-KVM